WordPress Security Benchmark

Full Stack Hardening Guide — Current Supported WordPress Releases on Linux (Ubuntu/Debian)

Version 1.1.1 — June 17, 2026

Contents

Table of Contents 2
Overview . . . 2
Target Technology . . . 2
Profile Definitions . . . 2
Assessment Status . . . 3
1.0 Web Server Configuration . . . 3
2.0 PHP Configuration . . . 10
3.0 Database Configuration . . . 14
4.0 WordPress Core Configuration . . . 18
5.0 Authentication and Access Control . . . 27
6.0 File System Permissions . . . 42
7.0 Logging and Monitoring . . . 46
8.0 Supply Chain and Component (Plugin and Theme) Management . . . 49
9.0 Web Application Firewall . . . 54
10.0 Backup and Recovery . . . 55
11.0 AI Integration Security . . . 56
12.0 Server Access and Network . . . 62
13.0 Multisite Security . . . 67
Appendix A: Recommendation Summary . . . 69
Cross-Document Control Classification Matrix . . . 71
Appendix B: Deprecated and Invalid Constants Guardrail . . . 72
Related Documents . . . 72
License and Attribution . . . 73

Table of Contents

Overview

The *WordPress Security Benchmark* provides prescriptive guidance for establishing a secure configuration posture for current supported WordPress releases running on a Linux server stack. It covers the full stack: the operating system firewall, web server (Nginx or Apache), PHP runtime, MySQL/MariaDB database, and the WordPress application layer.

The Benchmark is intended for system administrators, security engineers, DevOps teams, and WordPress developers responsible for deploying and maintaining WordPress installations in enterprise environments. It draws on many WordPress, LEMP/LAMP, and related security resources and standards (e.g., OWASP and NIST), as well as field experience with enterprise WordPress hardening.

Target Technology

- The latest WordPress release. As of October 7, 2026 that is WordPress 7.1.3 (7.1 was released on August 19, 2026). Only the most recent release is actively supported; security fixes for older branches (currently back to 4.7) are courtesy backports, not support. The controls in this edition were verified against WordPress 7.0 and reviewed for changes in 7.1. WordPress 7.2 is scheduled for December 8, 2026 and is not covered.
- Ubuntu 22.04+ / Debian 12+ (or equivalent RHEL/CentOS)
- Nginx 1.24+ or Apache 2.4+
- PHP 8.3+ (validate PHP 8.4 in staging before production rollout)
- MySQL 8.0+ or MariaDB 10.6+

Note on Containerization: While this benchmark assumes a traditional Linux stack, the principles and many of the configuration settings apply equally to containerized environments (Docker, Kubernetes). In such cases, configurations should be injected via environment variables or secret management systems rather than direct file edits where possible.

Profile Definitions

This benchmark defines two configuration profiles:

Level	Description
Level 1	Essential security settings that can be implemented on any WordPress deployment with minimal impact on functionality or performance. These form a baseline security posture that every site should meet. Implementing Level 1 items should not significantly inhibit the usability of the technology.
Level 2	Defense-in-depth settings intended for high-security environments. These recommendations may restrict functionality, require additional tooling, or involve operational overhead. They are appropriate for sites handling sensitive data, regulated industries, or high-value targets.

Assessment Status

Automated: Compliance can be verified programmatically using command-line tools, configuration file inspection, or API queries.

Manual: Compliance requires human judgment, review of policies, or inspection of settings through a graphical interface.

1.0 Web Server Configuration

This section provides recommendations for hardening the web server (Nginx or Apache) that serves the WordPress application.

1.1 Ensure TLS 1.2+ is enforced **Profile Applicability: Level 1**

Assessment Status: Automated

Description: Only TLS 1.2 and TLS 1.3 should be accepted. Level 1 applies to every deployment where the operator controls the web server. Modern web servers (Nginx 1.23.4+, Apache 2.4.x) default to TLS 1.2+. TLS 1.0 and 1.1 contain known vulnerabilities and must be disabled.

Rationale: TLS 1.0 and 1.1 are vulnerable to BEAST, POODLE, and other attacks. All major browsers have dropped support for these protocols. Enforcing 1.2+ eliminates a class of protocol-level attacks.

Impact: Legacy clients that do not support TLS 1.2 will be unable to connect. This is an acceptable trade-off for security.

Audit:

For Nginx, verify the ssl_protocols directive:

```
$ grep -r 'ssl_protocols' /etc/nginx/
```

Verify that the output contains only TLSv1.2 and TLSv1.3. For Apache:

```
$ grep -r 'SSLProtocol' /etc/apache2/
```

Verify the output shows `all -SSLv3 -TLSv1 -TLSv1.1` or equivalent.

Remediation:

For Nginx, set in the server or http block:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

For Apache, set in the VirtualHost or global config:

```
SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1
```

Restart the web server after changes.

Default Value: Nginx (1.23.4+): `TLSv1.2 TLSv1.3`. Apache 2.4: `SSLProtocol all -SSLv3` (TLSv1, TLSv1.1, TLSv1.2, and TLSv1.3 enabled; SSLv3 disabled). The remediation above explicitly disables TLSv1 and TLSv1.1 for both servers.

References:

- Mozilla SSL Configuration Generator

1.2 Ensure HTTP security headers are configured **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The web server should send security-related HTTP headers including Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security (HSTS), Referrer-Policy, and Permissions-Policy.

Rationale: HTTP security headers instruct the browser to enable built-in protections against common attacks such as XSS, clickjacking, MIME-type confusion, and insecure referrer leakage.

Impact: Overly restrictive Content-Security-Policy headers may break inline scripts, third-party integrations, or analytics tools. Note that `unsafe-inline` is often required for WordPress themes and plugins, but it represents a security trade-off. For Level 2, aim to remove `unsafe-inline` by using nonces or hashes.

Audit:

For Nginx, inspect the response headers:

```
$ curl -sI https://example.com | grep -iE '(content-security|x-content-type|x-frame|strict-transport|referrer-policy|permissions-policy)'
```

Verify all six headers are present.

Remediation:

For Nginx, add to the server block:

```
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header Permissions-Policy "geolocation=(), camera=(), microphone=()" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';" always;
```

In WordPress 7.1 and later, the block editor processes images in the browser using a Web Worker loaded from a `blob:` URL. A policy without `worker-src 'self' blob:` blocks that worker and WordPress falls back to server-side processing, so the stricter policy above remains safe to deploy. Add `worker-src 'self' blob:` only if client-side media processing is wanted. For Apache, use the Headers module:

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
```

Default Value: No security headers are set by default.

References:

- MDN Web Docs — HTTP Security Headers
- OWASP Secure Headers Project

1.3 Ensure server tokens and version information are hidden **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The web server should not disclose its version number, operating system, or module information in HTTP response headers or error pages.

Rationale: Version information helps attackers identify specific vulnerabilities to target. Removing it forces attackers to probe the server more actively, increasing the chance of detection.

Impact: Hiding server and version metadata has no impact on normal functionality. Troubleshooting server issues may require slightly more effort, as administrators must check local configuration rather than relying on HTTP response headers.

Audit:

```
$ curl -sI https://example.com | grep -i 'server'
```

Verify the Server header does not contain version numbers.

Remediation:

For Nginx:

```
server_tokens off;
```

For Apache:

```
ServerTokens Prod
ServerSignature Off
```

Default Value: Nginx: server_tokens on (version exposed). Apache: ServerTokens Full.

References:

- Apache `ServerTokens` Directive
- Apache `ServerSignature` Directive
- Nginx `server_tokens` Directive

1.4 Ensure direct PHP execution is blocked in upload directories Profile Applicability: Level 1

Assessment Status: Automated

Description: PHP execution must be disabled in the wp-content/uploads/ directory and any other directories intended only for static file storage.

Rationale: If an attacker uploads a malicious PHP file through a vulnerability (e.g., an insecure file upload in a plugin), blocking PHP execution in the uploads directory prevents the file from being executed.

Impact: None. Legitimate WordPress operations never require PHP execution from the uploads directory.

Audit:

For Nginx, verify a location block exists for uploads:

```
$ grep -A5 'uploads' /etc/nginx/sites-enabled/*
```

Verify that PHP processing is denied for the uploads directory. The presence of the block is not sufficient: Nginx uses the first regular-expression location that matches, in the order the locations appear in the configuration, so the denial has no effect if a generic `location ~ \.php$` block comes first. Inspect the effective configuration (`nginx -T`) for ordering, then confirm the behavior with an inert test file on a staging copy:

```
$ echo '<?php echo "executed";' | sudo tee /path/to/wordpress/wp-content/uploads/exec-test.php
$ curl -s -o /dev/null -w '%{http_code}\n' https://example.com/wp-content/uploads/exec-test.php
$ sudo rm /path/to/wordpress/wp-content/uploads/exec-test.php
```

A 403 response indicates PHP execution is denied. A 200 response containing `executed` indicates the control is not effective.

Remediation:

For Nginx, add to the server block, above any generic PHP handler such as `location ~ \.php$` (regular-expression locations are evaluated in order and the first match wins):

```
location ~* /wp-content/uploads/.*\.php$ {
  deny all;
}
```

Adjust the path if uploads are stored elsewhere, and extend the pattern to any other extensions the server passes to PHP (for example, `.phtml` or `.phar`). For Apache, create wp-content/uploads/.htaccess:

```
<FilesMatch "\.php$">
  Require all denied
</FilesMatch>
```

Default Value: PHP execution is allowed in all directories by default.

References:

- WordPress Hardening — File Permissions

1.5 Ensure rate limiting is configured for all API surfaces **Profile Applicability: Level 1**

Assessment Status: Automated

Description: HTTP request rate limiting should be applied to all authentication and API endpoints, including `wp-login.php`, `xmlrpc.php`, and the WordPress REST API (`/wp-json/`).

Rationale: WordPress authentication and API interfaces are primary targets for automated brute-force and resource exhaustion attacks — classified as API4: Unrestricted Resource Consumption in the OWASP API Security Top 10 (2023). Comprehensive rate limiting across all entry points reduces the effectiveness of these attacks and protects server resources.

Impact: Aggressive rate limiting may lock out legitimate users or break third-party integrations (e.g., decoupled front-ends) if not configured with appropriate burst allowances and allowlist exceptions.

Audit:

For Nginx, check for `limit_req` configuration:

```
$ grep -r 'limit_req' /etc/nginx/
```

Verify rate limiting zones are defined and applied to login, XML-RPC, and REST API locations. The REST API is reachable in two forms: the pretty-permalink path (`/wp-json/...`) and the query-string form (`/?rest_route=/...`), which works on every site. Confirm both are covered.

Remediation:

For Nginx, define rate limiting zones and apply them to the relevant locations:

```
# In http block:
limit_req_zone $binary_remote_addr zone=wplogin:10m rate=1r/s;
limit_req_zone $binary_remote_addr zone=wpapi:10m rate=5r/s;

# In server block:
location = /wp-login.php {
    limit_req zone=wplogin burst=3 nodelay;
    # ... PHP processing ...
}

location = /xmlrpc.php {
    limit_req zone=wplogin burst=3 nodelay;
    # ... PHP processing ...
}

location ~ ^/wp-json/ {
    limit_req zone=wpapi burst=10 nodelay;
    # ... PHP processing ...
}
```

The `/wp-json/` location does not match REST requests sent as `/?rest_route=/...`. To cover them, key a second zone on a variable that is empty for other requests (Nginx does not count requests with an empty key):

```
# In http block:
map $arg_rest_route $rest_route_key {
    default $binary_remote_addr;
    ""      "";
}
limit_req_zone $rest_route_key zone=wpapiquery:10m rate=5r/s;

# In the location that handles the front controller (index.php):
limit_req zone=wpapiquery burst=10 nodelay;
```

Default Value: No rate limiting is configured by default.

References:

- OWASP API4: Unrestricted Resource Consumption
- Nginx `limit_req` Module
- WordPress REST API FAQ

2.0 PHP Configuration

This section provides recommendations for securing the PHP runtime environment.

> **NOTE:** Audit the PHP runtime that serves the site, not the command-line interpreter. The php CLI binary can be a different PHP version and loads its own `php.ini`, so `php -i` can report hardened values while PHP-FPM is not hardened. The audits below query the PHP-FPM binary (adjust `php-fpm8.3` to the installed version; under Apache `mod_php`, read the values from the Apache SAPI's `php.ini` instead). PHP-FPM pool files can override `php.ini` with `php_admin_value` and `php_value`, so also check the pool that serves the site:

```
$ grep -RE 'php_(admin_)?(value|flag)' /etc/php/8.3/fpm/pool.d/
```

2.1 Ensure expose_php is disabled **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The expose_php directive in php.ini must be set to Off. This prevents PHP from disclosing its presence and version in HTTP response headers (X-Powered-By).

Rationale: Version information assists attackers in identifying vulnerabilities specific to the running PHP version.

Impact: None on normal functionality. Automated server scanners or troubleshooting tools will not automatically detect the PHP version from response headers.

Audit:

```
$ sudo php-fpm8.3 -i | grep expose_php
```

Verify the output shows `expose_php => Off => Off`.

Remediation:

In php.ini:

```
expose_php = Off
```

Restart PHP-FPM or the web server.

Default Value: expose_php = On

References:

- PHP `expose_php` Directive
- OWASP PHP Configuration Cheat Sheet

2.2 Ensure display_errors is disabled in production **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The display_errors directive must be set to Off in production environments. PHP errors should be logged to a file, not displayed to users.

Rationale: Displayed PHP errors can reveal file paths, database connection details, and application structure to attackers.

Impact: Developers cannot see errors directly in the browser when debugging in production. All errors must be reviewed via server error logs.

Audit:

```
$ sudo php-fpm8.3 -i | grep display_errors
```

Verify: `display_errors => Off => Off`.

Remediation:

In php.ini:

```
display_errors = Off
log_errors = On
error_log = /var/log/php/error.log
```

Default Value: display_errors = On in development configurations.

References:

- PHP `display_errors` Configuration
- PHP Error Handling Basics
- WordPress `wp-config.php` API

2.3 Ensure dangerous PHP functions are disabled **Profile Applicability: Level 1**

Assessment Status: Automated

Description: PHP functions that allow arbitrary command execution, code evaluation, or information disclosure should be disabled unless specifically required.

Rationale: If an attacker achieves code execution (e.g., through a vulnerable plugin), these functions enable them to execute system commands, read arbitrary files, or escalate the attack.

Impact: Some WordPress plugins may require specific functions. Test thoroughly before deploying. The `eval()` function is a language construct and cannot be disabled via `disable_functions`.

Recommendation (Level 2): For high-security environments, consider using a PHP security extension like **Snuffleupagus** to mitigate `eval()` and provide additional hardening that `disable_functions` cannot achieve.

Audit:

```
$ sudo php-fpm8.3 -i | grep disable_functions
```

Verify the output includes dangerous functions.

Remediation:

In php.ini:

```
disable_functions = exec,passthru,shell_exec,system,proc_open,popen,curl_multi_ex
```

Default Value: No functions are disabled by default.

References:

- PHP `disable_functions` Directive
- OWASP PHP Configuration Cheat Sheet

2.4 Ensure open_basedir restricts file access Profile Applicability: Level 2

Assessment Status: Automated

Description: The open_basedir directive should restrict PHP file operations to the WordPress installation directory and required system paths only.

Rationale: open_basedir prevents PHP code from reading or writing files outside the defined directory tree, limiting the impact of a file inclusion or traversal vulnerability.

Impact: Must include the WordPress root, /tmp (for file uploads), and the PHP session directory. Incorrect configuration will break WordPress.

Audit:

```
$ sudo php-fpm8.3 -i | grep open_basedir
```

Verify a restricted path is configured.

Remediation:

In the PHP-FPM pool configuration or php.ini:

```
open_basedir = /var/www/example.com:/tmp:/usr/share/php
```

Default Value: open_basedir is not set (unrestricted).

References:

- PHP `open_basedir` Directive
- OWASP PHP Configuration Cheat Sheet

2.5 Ensure PHP session security is configured Profile Applicability: Level 1

Assessment Status: Automated

Description: PHP session configuration should enforce secure defaults: cookies marked Secure, HttpOnly, and SameSite=Lax or Strict. Session ID entropy and hashing should use strong algorithms.

Rationale: Secure session configuration prevents session fixation, cookie theft via XSS, and cross-site request forgery via session cookies. Note: WordPress core does not use PHP native sessions (`$_SESSION`) — it implements its own authentication via cookies and database-stored session tokens. These PHP session settings are defense-in-depth for plugins that call `session_start()`.

Impact: Users must connect via HTTPS to maintain a session. Legacy applications or intra-server tools attempting to access sessions over HTTP will be unable to maintain state.

Audit:

```
$ sudo php-fpm8.3 -i | grep -E 'session\.(cookie_secure|cookie_httponly|cookie_same
```

Verify all are set to appropriate secure values.

Remediation:

In php.ini:

```
session.cookie_secure = 1
session.cookie_httponly = 1
session.cookie_samesite = Lax
session.use_strict_mode = 1
session.use_only_cookies = 1
```

Default Value: session.cookie_secure = 0, session.cookie_httponly = 0 (insecure defaults).

References:

- PHP Session Security Configuration

3.0 Database Configuration

This section covers MySQL/MariaDB configuration relevant to WordPress security.

3.1 Ensure the WordPress database user has minimal privileges **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The MySQL/MariaDB user account used by WordPress should be scoped to the WordPress database only and granted the minimum privileges required for normal operation and maintenance. WordPress core, plugins, themes, and updates require eight privileges: SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, and DROP. The WordPress project explicitly recommends retaining all eight to avoid failed updates and data loss. Never grant FILE, PROCESS, SUPER, GRANT OPTION, or ALL PRIVILEGES.

Rationale: Granting excessive privileges (e.g., FILE, SUPER, GRANT) increases the impact of a SQL injection vulnerability. With minimal privileges, an attacker who achieves SQLi cannot read arbitrary files, modify grants, or perform administrative operations. Restricting the user to a single database prevents lateral movement to other databases on the same server.

Impact: Revoking CREATE, ALTER, INDEX, or DROP can break plugin activations, WordPress core updates, and database migrations. If a high-security environment requires restricting schema-modification privileges, use a two-account model: a limited account (SELECT, INSERT, UPDATE, DELETE) for runtime and a privileged account (adding CREATE, ALTER, INDEX, DROP) used only during maintenance windows. This approach requires changing `DB_USER` in `wp-config.php` before and after each update cycle and is only practical for organizations with formal change-management processes.

Audit:

Run as the MySQL root user:

```
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR 'wp_user'@'localhost';
```

Verify the user has privileges only on the WordPress database and only the required types.

Remediation:

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'wp_user'@'localhost';
```

This form is documented to remove privileges at every level (global, database, table, column, and routine). It does not revoke roles granted to the account, so confirm the result with `SHOW GRANTS` after applying the new grant.

```
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, DROP ON wp_database.* TO
FLUSH PRIVILEGES;
```

Default Value: Depends on initial setup. Many installation guides grant ALL PRIVILEGES.

References:

- WordPress Hardening — Database Security
- MySQL Privilege System

3.2 Ensure the database is not accessible from external hosts **Profile Applicability: Level 1**

Assessment Status: Automated

Description: MySQL/MariaDB should be configured to listen only on localhost (127.0.0.1) or a Unix socket. Remote TCP connections should be disabled unless required and tunneled through SSH or a VPN.

Rationale: A database accessible over the network expands the attack surface. Brute-force attacks, credential stuffing, and exploitation of database vulnerabilities become possible from any host that can reach the port.

Impact: Remote database administration tools cannot connect directly unless tunneled through SSH or a VPN, imposing additional operational requirements for database administrators.

Audit:

```
$ grep -E 'bind-address|skip-networking' /etc/mysql/mysql.conf.d/mysqld.cnf
```

Verify bind-address is 127.0.0.1 or ::1.

```
$ ss -tlnp | grep 3306
```

Verify MySQL is listening only on 127.0.0.1:3306.

Remediation:

In mysqld.cnf or my.cnf under [mysqld]:

```
bind-address = 127.0.0.1
```

Restart MySQL.

Default Value: bind-address = 0.0.0.0 (listening on all interfaces) on some distributions.

References:

- WordPress Hardening — Database Security
- MySQL Security Guidelines
- MySQL `bind_address` Variable

3.3 Review database table prefix strategy (optional obscurity control) Profile Applicability: Level 2

Assessment Status: Manual

Description: A non-default database table prefix may be used as an optional defense-in-depth control, but it should not be treated as a primary hardening measure.

Rationale: WordPress hardening guidance classifies table-prefix changes as a form of security through obscurity with limited security value. Prioritize patching, least privilege, and secure coding controls before prefix customization.

Impact: Changing the prefix on an existing installation requires updating table names and option/usermeta values and can break plugins if done incorrectly.

Audit:

Inspect `wp-config.php`:

```
$ grep 'table_prefix' /path/to/wp-config.php
```

Record whether the installation uses the default `wp_` prefix and ensure the choice is documented in deployment standards.

Remediation:

Keep the default prefix unless your environment has a documented policy requiring customization. If customization is required, set it only during initial provisioning and test plugin compatibility:

```
$table_prefix = 'wxyz_';
```

Default Value: `$table_prefix = 'wp_';`

References:

- WordPress `wp-config.php` — `table_prefix`
- WordPress Hardening — Security through Obscurity

3.4 Ensure database query logging is enabled **Profile Applicability: Level 2**

Assessment Status: Automated

Description: MySQL/MariaDB general query log or slow query log should be enabled to support forensic analysis and intrusion detection.

Rationale: Query logs provide critical evidence during incident response, including the exact queries executed by an attacker who achieved SQL injection. They also help identify performance issues that may indicate abuse.

Impact: General query logging incurs significant I/O overhead and should be used selectively or only during investigations. Slow query logging has minimal overhead and can remain enabled.

Audit:

```
$ grep -E '(general_log|slow_query_log)' /etc/mysql/mysql.conf.d/mysqld.cnf
```

Verify at minimum slow_query_log is enabled.

Remediation:

In mysqld.cnf under [mysqld]:

```
slow_query_log = 1
slow_query_log_file = /var/log/mysql/mysql-slow.log
long_query_time = 2
```

For investigations, temporarily enable:

```
general_log = 1
general_log_file = /var/log/mysql/mysql-general.log
```

Default Value: Both logs are disabled by default.

References:

- MySQL General Query Log
- MySQL Log Destinations

4.0 WordPress Core Configuration

This section covers security settings in `wp-config.php` and WordPress core behavior.

4.1 Ensure `DISALLOW_FILE_EDIT` is set to true (baseline) Profile Applicability: Level 1

Assessment Status: Automated

Description: The `DISALLOW_FILE_EDIT` constant should be defined as `true` in `wp-config.php` to disable the built-in theme and plugin editor.

Note: `DISALLOW_FILE_MODS` is a stricter superset that also blocks plugin/theme installation and updates from the Dashboard. Use `DISALLOW_FILE_MODS` only for hardened profiles with a documented external update workflow.

Rationale: The built-in editor is a common post-compromise persistence vector after admin-account takeover. Disabling it reduces risk without blocking normal update paths.

Impact: Administrators lose browser-based code editing in `wp-admin`. Theme/plugin updates continue to work unless `DISALLOW_FILE_MODS` is also enabled.

Audit:

```
$ grep 'DISALLOW_FILE_EDIT' /path/to/wp-config.php
```

Verify: `define( 'DISALLOW_FILE_EDIT', true );`

If `DISALLOW_FILE_MODS` is also set, verify deployment automation handles plugin/theme/core updates.

Remediation:

Add to `wp-config.php` before 'That's all, stop editing!':

```
define( 'DISALLOW_FILE_EDIT', true );
```

Optional hardened profile:

```
define( 'DISALLOW_FILE_MODS', true );
```

Default Value: Not set (editor enabled).

References:

- WordPress Hardening — Disable File Editing
- WordPress Hardening

4.2 Ensure `FORCE_SSL_ADMIN` is set to true **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The `FORCE_SSL_ADMIN` constant forces all admin and login pages to be served over HTTPS.

Rationale: Without this setting, admin session cookies could be transmitted over unencrypted HTTP if a user accesses the admin via an HTTP URL, enabling session hijacking via network interception.

Impact: All users must access the Dashboard over HTTPS. If the site does not have a valid TLS/SSL certificate configured, the admin interface will become inaccessible.

Audit:

```
$ grep 'FORCE_SSL_ADMIN' /path/to/wp-config.php
```

Verify: `define( 'FORCE_SSL_ADMIN', true );`

Remediation:

Add to `wp-config.php`:

```
define( 'FORCE_SSL_ADMIN', true );
```

Default Value: Not set (HTTPS not enforced for admin).

References:

- WordPress `wp-config.php` — `FORCE_SSL_ADMIN`
- WordPress HTTPS Administration
- `force_ssl_admin()` Function Reference

4.3 Ensure WordPress debug mode is disabled in production **Profile Applicability: Level 1**

Assessment Status: Automated

Description: `WP_DEBUG` must be set to `false` in production environments. `WP_DEBUG_DISPLAY` must also be `false`, and `WP_DEBUG_LOG` should write to a non-public location if enabled.

Rationale: Debug output can reveal file paths, database queries, and PHP errors to attackers. Debug log files in the default location (`wp-content/debug.log`) are publicly accessible unless explicitly blocked.

Impact: Suppresses error visibility for administrators and developers on the live site, requiring them to consult the server's error log files or a dedicated logging tool to diagnose issues.

Audit:

```
$ grep -E 'WP_DEBUG|WP_DEBUG_DISPLAY|WP_DEBUG_LOG' /path/to/wp-config.php
```

Verify `WP_DEBUG` and `WP_DEBUG_DISPLAY` are `false`. If `WP_DEBUG_LOG` is enabled, verify the log path is outside the web root or blocked by the web server.

Remediation:

```
define( 'WP_DEBUG', false );
define( 'WP_DEBUG_DISPLAY', false );
define( 'WP_DEBUG_LOG', false );
```

If logging is needed, direct to a non-public path:

```
define( 'WP_DEBUG_LOG', '/var/log/wordpress/debug.log' );
```

Default Value: `WP_DEBUG = false` (secure by default). However, many deployment guides enable debug mode.

References:

- WordPress `wp-config.php` — `WP_DEBUG`
- `wp_debug_mode()` Function Reference
- WordPress `wp-config.php` API

4.4 Ensure XML-RPC is disabled **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The XML-RPC interface (`xmlrpc.php`) should be blocked at the web server unless specifically required by a remote publishing client or integration.

Rationale: XML-RPC is a second authentication surface that accepts username and password on every request, and its pingback methods have been abused for DDoS reflection and internal port probing. Historically, the `system.multicall` method allowed hundreds of password attempts in a single HTTP request; since WordPress 4.4, core stops evaluating credentials for the rest of a request after the first authentication failure, so that amplification no longer works on current versions. Password guessing across separate requests and pingback abuse remain. While WordPress core mitigates XML eXternal Entity (XXE) and entity expansion attacks by disabling the loading of custom XML entities, disabling XML-RPC entirely removes the endpoint from the attack surface.

Impact: Disabling XML-RPC will break Jetpack (which requires it for WordPress.com communication), the WordPress mobile app (older versions), and any third-party tool that uses the XML-RPC API.

Audit:

```
$ curl -s -o /dev/null -w '%{http_code}' -X POST -H 'Content-
Type: text/xml' --data '<?xml version="1.0"?><methodCall><methodName>system.listMe
```

A 200 response indicates XML-RPC is accessible. A 403 or 404 indicates it is blocked. Use a POST request: an enabled endpoint answers a plain GET with 405, which does not show whether it is blocked.

Remediation:

Block at the web server level. This is the only complete block. For Nginx:

```
location = /xmlrpc.php {
  deny all;
}
```

Where the web server cannot be configured, a must-use plugin (placed in `wp-content/mu-plugins/`, not `wp-config.php`) is a partial measure and does not satisfy this control on its own:

```
<?php
// Disables XML-RPC methods that require authentication.
add_filter( 'xmlrpc_enabled', '__return_false' );

// Remove pingback methods, which do not require authentication.
add_filter( 'xmlrpc_methods', function( $methods ) {
   unset( $methods['pingback.ping'], $methods['pingback.extensions.getPingbacks']
    return $methods;
} );
```

The `xmlrpc_enabled` filter turns off only the methods that require authentication. Pingbacks and any other unauthenticated or plugin-registered methods remain available unless removed separately, and the endpoint still loads WordPress for every request.

Additionally, disable trackbacks and pingbacks in **Settings Discussion** by unchecking "Allow link notifications from other blogs (pingbacks and trackbacks) on new posts." Trackbacks operate independently of `xmlrpc.php` and should be disabled separately.

Default Value: XML-RPC, trackbacks, and pingbacks are all enabled by default.

References:

- WordPress Hardening
- `xmlrpc_enabled` Hook
- `xmlrpc_methods` Hook
- `wp_xmlrpc_server::login()`

4.5 Ensure automatic core updates are enabled **Profile Applicability: Level 1**

Assessment Status: Automated

Description: WordPress automatic background updates for minor (security) releases must remain enabled. Major version auto-updates should be evaluated based on organizational policy.

Rationale: Minor releases contain only security fixes and critical bug patches. Disabling them leaves the site vulnerable to known, publicly disclosed exploits.

Impact: In rare cases, a minor update may introduce a regression. Managed hosting providers typically handle this with rollback capabilities.

Audit:

```
$ wp config get WP_AUTO_UPDATE_CORE --path=/path/to/wordpress 2>/dev/null
$ grep 'WP_AUTO_UPDATE_CORE\|AUTOMATIC_UPDATER_DISABLED' /path/to/wp-config.php
```

Verify `WP_AUTO_UPDATE_CORE` is not set to `false` and `AUTOMATIC_UPDATER_DISABLED` is not `true`.

Remediation:

Ensure `wp-config.php` does not contain:

```
define( 'AUTOMATIC_UPDATER_DISABLED', true );
```

Optionally, explicitly enable minor updates:

```
define( 'WP_AUTO_UPDATE_CORE', 'minor' );
```

Default Value: Minor auto-updates are enabled by default since WordPress 3.7.

References:

- WordPress Auto-Update Configuration

4.6 Ensure unique authentication keys and salts are configured **Profile Applicability: Level 1**

Assessment Status: Automated

Description: All eight authentication keys and salts in `wp-config.php` must be set to unique, random values. These are: `AUTH_KEY`, `SECURE_AUTH_KEY`, `LOGGED_IN_KEY`, `NONCE_KEY`, and their corresponding `SALT` counterparts.

Rationale: These keys are used to hash session tokens stored in cookies. Default, empty, or guessable values weaken cookie security, making session forgery and hijacking easier.

Impact: If keys are changed on an existing site, all currently logged-in users will be immediately logged out and forced to re-authenticate, as their existing session cookies will become invalid.

Audit:

Count the definitions without printing the secret values:

```
$ grep -cE "define\( *'(AUTH_KEY|SECURE_AUTH_KEY|LOGGED_IN_KEY|NONCE_KEY|AUTH_SALT
config.php
$ grep -c 'put your unique phrase here' /path/to/wp-config.php
```

Verify the first count is 8 and the second is 0 (the placeholder value is not in use). Do not print the keys and salts to a terminal, log, ticket, or AI-assisted session; if the values themselves must be inspected for length and uniqueness, do so locally on the host.

Remediation:

Generate new keys using the WordPress.org API:

```
$ curl -s https://api.wordpress.org/secret-key/1.1/salt/
```

Replace the key definitions in `wp-config.php` with the generated output.

Default Value: Placeholder values ('put your unique phrase here') in fresh installations.

References:

- WordPress Hardening — Security Keys

4.7 Consider replacing `wp-cron.php` with a system cron job **Profile Applicability: Level 2**

Assessment Status: Automated

Description: For higher operational reliability, WordPress's built-in pseudo-cron (`wp-cron.php`) can be disabled and replaced with a real system-level cron job. This improves scheduling predictability, especially on very low-traffic or very high-traffic sites.

Rationale: `wp-cron.php` is traffic-triggered, so jobs can be delayed on low-traffic sites and bursty on high-traffic sites. A system cron schedule is deterministic. This is primarily a reliability/operations control with secondary attack-surface benefits when direct `wp-cron.php` access is blocked.

Impact: Disabling `wp-cron.php` without configuring a system cron replacement will prevent scheduled tasks (e.g., publishing scheduled posts, checking for updates, sending email digests) from running.

Audit:

```
$ grep 'DISABLE_WP_CRON' /path/to/wp-config.php
```

Verify: `define( 'DISABLE_WP_CRON', true );`

Verify a system cron job is configured:

```
$ crontab -l | grep -E 'wp.cron'
```

Verify the entry uses `wp cron event run --due-now` (WP-CLI), not `curl` to `wp-cron.php`. Verify `wp-cron.php` is blocked at the web server level:

```
$ curl -s -o /dev/null -w '%{http_code}' https://example.com/wp-cron.php
```

A 403 response confirms the endpoint is blocked.

Remediation:

1. Add to `wp-config.php`:

```
define( 'DISABLE_WP_CRON', true );
```

2. Add a system cron job (runs every 5 minutes) using WP-CLI:

```
*/5 * * * * cd /path/to/wordpress && wp cron event run --due-now > /dev/null 2>&1
```

3. Block direct external access to `wp-cron.php` at the web server level (Nginx):

```
location = /wp-cron.php {
    deny all;
}
```

This is safe because WP-CLI executes PHP directly and does not use HTTP.

> For the complete operational procedure — crontab installation, web server blocking, expected output, rollback, and escalation — see WordPress Operations Runbook §6.6.

Default Value: `wp-cron.php` is enabled and triggered on every page load by default.

References:

- Hooking WP-Cron into the System Task Scheduler
- WP-Cron Overview

5.0 Authentication and Access Control

This section addresses user authentication, session management, and role-based access control within WordPress.

5.1 Ensure two-factor authentication is required for administrators **Profile Applicability: Level 1**

Assessment Status: Manual

Description: All user accounts with the Administrator role must have two-factor authentication (2FA) enabled using TOTP-based authenticator apps or hardware security keys (WebAuthn/FIDO2).

Rationale: Compromised administrator credentials grant full control over the WordPress installation. 2FA ensures that a stolen password alone is insufficient to gain access.

Impact: Requires a 2FA plugin. This documentation set standardizes on `two-factor` for operational consistency. Equivalent enterprise-approved alternatives may be used when requirements are met. WordPress core does not include mandatory 2FA natively.

Audit:

This is a manual check. Verify that: 1. A 2FA plugin is installed and active. 2. Every administrator account has 2FA configured. Check each account; an active plugin does not show that any user has enrolled. With `two-factor` 0.17 or later: `wp two-factor status <user>`. 3. A control is in place that prevents an unenrolled administrator from using the Dashboard, and it has been tested with an unenrolled test account. 4. SMS-based 2FA is not used (vulnerable to SIM-swapping).

Remediation:

Install and configure `two-factor` (or an approved equivalent). Require 2FA enrollment for all users with Administrator, Editor, or Shop Manager roles. The `two-factor` plugin provides enrollment and the login challenge but has no built-in setting that makes 2FA mandatory for a role. Enforcement requires an additional control: MFA enforced at a single sign-on identity provider, an approved plugin that enforces enrollment, or maintained custom code. Document which one is used. Recommended: Enforce 2FA as mandatory for admin roles with a grace period for initial setup.

> For step-by-step WP-CLI installation, configuration, break-glass recovery, and operational verification of the `two-factor` plugin, see WordPress Operations Runbook §5.5.

Default Value: No 2FA is configured by default.

References:

- NIST SP 800-63B — Multi-Factor Authentication
- WordPress Developer Documentation › Advanced Administration Handbook › Security › Hardening › MFA
- Two-Factor by WordPress.org - Community-maintained plugin

5.2 Ensure the number of administrator accounts is minimized **Profile Applicability: Level 1**

Assessment Status: Manual

Description: The number of user accounts with the Administrator role should be limited to the minimum required. A primary administrator account should be reserved for emergency use only. Administrator account usernames must not use easily guessed values such as `admin`, `administrator`, or `webmaster`, which are the first targets of automated brute-force attacks. Administrator accounts must not publish content or comment while logged in. Whoever publishes gets an Author or Editor account.

Rationale: Each administrator account is a potential entry point. Compromising any single admin account grants full site control. Minimizing admin accounts reduces the attack surface. Default or predictable usernames make brute-force and credential-stuffing attacks significantly easier. Publishing makes an account visible: its display name appears in bylines, feeds and comments, and its slug in author URLs (see 5.9). When the publishing account is an administrator, every post advertises the account most worth attacking. When publishing is done from a lower-privilege account, an exposed identity leads to an account that cannot take over the site.

Impact: Concentrates administrative privileges among fewer users. Operations that require administrator access may experience delays if authorized personnel are not immediately available.

Audit:

```
$ wp user list --role=administrator --fields=ID,user_login,user_email --path=/path/to/wordpress
```

Review the list. Verify that each admin account is actively needed and assigned to a specific individual.

Count published content owned by administrator accounts across every public post type. Any result above zero is a finding. Do not use `--post_type=any`: WP-CLI passes it to `WP_Query`, which then skips every post type registered with `exclude_from_search`, including public types with bylines such as bbPress topics and replies. Do not filter by `post_type_supports( 'author' )` either: plugins can store a public author without declaring that support, and core declares it for site-building types that administrators are expected to own, such as templates, template parts, synced patterns and Customizer changesets. Attachments are left out here and reviewed in remediation step 6. These are site-level checks: `--role=administrator` lists one site's Administrators, not a network's Super Admins. On Multisite, run them on each site with `--url=<site-url>`, and audit Super Admins with control 13.1.

```
$ types=$(wp eval 'echo implode( ",", array_diff( get_post_types( array( "public" =>
-path=/path/to/wordpress)
$ admins=$(wp user list --role=administrator --format=ids --
path=/path/to/wordpress | tr ' ' ',')
$ wp post list --author="$admins" --post_type="$types" --post_status=publish -
-format=count --path=/path/to/wordpress
```

The audit measures ownership, which is what the public record shows. It cannot show who clicked Publish, so an administrator publishing a post owned by another account passes.

Remediation:

1. Audit existing administrator accounts. 2. Replace any account using a predictable username (e.g., `admin`) with a unique, non-guessable login. WP-CLI does not support changing `user_login` in place:

```
$ wp user create <new-username> <new-user-email> --role=administrator -
-path=/path/to/wordpress
$ wp user delete <old-user-id> --reassign=<new-user-id> --yes --
path=/path/to/wordpress
```

3. Downgrade accounts that don't require full admin capabilities to Editor or a custom role. 4. Reserve one primary administrator account for break-glass emergencies. 5. Use custom roles with tailored capabilities for day-to-day operations. 6. Give each person who publishes a separate Author or Editor account, and reassign content that administrator accounts own. Use the `$types` list from the audit and name the statuses to move rather than relying on the default, which also matches attachments' `inherit` status and leaves trash and auto-drafts out. Capture the post IDs once, review that list, then reassign exactly those posts, so nothing changes between review and update. Review attachments separately:

```
$ statuses=publish,future,draft,pending,private
$ ids=$(wp post list --author=<admin-user-id> --post_type="$types" --
post_status="$statuses" --format=ids --path=/path/to/wordpress | tr ' ' ',')
$ wp post list --post__in="$ids" --post_type="$types" --post_status="$statuses" -
-fields=ID,post_type,post_status,post_title --path=/path/to/wordpress
$ wp post update $(echo "$ids" | tr ',' ' ') --post_author=<author-
user-id> --path=/path/to/wordpress
$ wp post list --author=<admin-user-id> --post_type=attachment --
post_status=inherit --fields=ID,post_title --path=/path/to/wordpress
```

Default Value: One administrator account is created during installation.

References:

- WordPress Roles and Capabilities
- WordPress Multisite Administration
- OWASP Authentication Cheat Sheet

5.3 Ensure maximum session lifetime is enforced **Profile Applicability: Level 1**

Assessment Status: Automated

Description: WordPress session cookies should have a maximum lifetime enforced, regardless of user activity. Privileged accounts (Administrators, Editors) should have shorter session limits (8–24 hours), including when "Remember Me" is selected. This control covers maximum lifetime only. Idle-session timeouts and purging sessions on role or capability changes are recommended additional measures; WordPress core provides neither, so they require a session-management plugin or custom code and must be verified separately.

Rationale: Long-lived sessions increase the window of opportunity for session hijacking. If an auth cookie is stolen, a shorter lifetime limits how long the attacker can use it. Idle session timeouts and scheduled session destruction provide additional layers of defense that reduce the exposure window even further.

Impact: Users will need to re-authenticate more frequently. This can be mitigated with trusted device verification.

Audit:

Check for session management plugins or custom code:

```
$ grep -r 'auth_cookie_expiration' /path/to/wp-content/mu-plugins/ /path/to/wp-config.php
```

Verify a filter is in place to limit session lifetime. The search shows only that a filter exists in those files. Confirm the enforced value by signing in as an administrator and checking the expiry of the `wordpress_logged_in_*` cookie, or the `expiration` of the newest entry in the user's `session_tokens` user meta.

Remediation:

Add a must-use plugin (wp-content/mu-plugins/session-limits.php):

```
add_filter( 'auth_cookie_expiration', function( $expiration, $user_id, $remember ) {
  // The same limit applies whether or not "Remember Me" was selected.
  if ( user_can( $user_id, 'manage_options' ) ) {
   return 8 * HOUR_IN_SECONDS; // 8 hours for administrators and Super Admins
  }

  return 24 * HOUR_IN_SECONDS; // 24 hours for others
}, 10, 3 );
```

Default Value: 48 hours (2 days) without 'Remember Me'; 14 days with 'Remember Me'.

References:

- `auth_cookie_expiration` Hook
- `wp_set_auth_cookie()` Function Reference
- OWASP Session Management Cheat Sheet

5.4 Ensure user enumeration is prevented **Profile Applicability: Level 1**

Assessment Status: Automated

Description: The REST API user endpoint and author archive URLs should be restricted to prevent unauthenticated enumeration of usernames.

Rationale: Username enumeration provides attackers with valid login targets for brute-force and credential stuffing attacks. The default WordPress REST API exposes user slugs at `/wp-json/wp/v2/users`, and author archives expose usernames via `/?author=N` URLs.

Impact: Blocking the REST API users endpoint may affect plugins that rely on it for public author data (e.g., some theme author bio features).

Audit:

```
$ curl -s https://example.com/wp-json/wp/v2/users | python3 -
m json.tool
```

If the response returns user data, enumeration is possible.

```
$ curl -sI https://example.com/?author=1
```

If the response is a 301 redirect to an author archive, enumeration is possible.

Remediation:

Require authentication for read access to the REST API users endpoint via a must-use plugin:

```
// Require authentication for read access to /wp/v2/users while preserving
// core's own permission checks for every operation.
add_filter( 'rest_endpoints', function( $endpoints ) {
    $routes = array( '/wp/v2/users', '/wp/v2/users/(?P<id>[\d]+)' );

    foreach ( $routes as $route ) {
        if ( empty( $endpoints[ $route ] ) ) {
            continue;
        }

        foreach ( $endpoints[ $route ] as $key => $handler ) {
        // Skip route options (namespace, schema, args); only handlers have numeric
        if ( ! is_numeric( $key ) || ! is_array( $handler ) || empty( $handler['permi
                continue;
            }

            // Leave create, update, and delete handlers untouched.
        $methods = is_array( $handler['methods'] ) ? array_keys( $handler['methods'
        if ( ! in_array( 'GET', array_map( 'trim', $methods ), true ) ) {
                continue;
            }

            $core_check = $handler['permission_callback'];
```

```
        $endpoints[ $route ][ $key ]['permission_callback'] = function( $request )
                if ( ! is_user_logged_in() ) {
                    return new WP_Error(
                        'rest_user_cannot_view',
                        'Authentication is required to view users.',
                array( 'status' => rest_authorization_required_code() )
                    );
                }

           // Defer to the original core check for authenticated requests.
                return call_user_func( $core_check, $request );
            };
        }
    }

    return $endpoints;
} );
```

This wraps core's permission check rather than replacing it or removing the routes. Anonymous read requests receive 401; authenticated requests are still decided by core, so the block editor's author lookups keep working for Editors, Authors, and Contributors, and the create, update, and delete handlers keep their original capability checks. Do not unset the users routes for every user without `list_users` (only Administrators have it by default), and do not replace the routes' permission callbacks with a single capability check. Block author archive enumeration at the web server level or with a plugin.

Default Value: User data is publicly accessible via the REST API and author archives.

References:

- OWASP Authentication Cheat Sheet — Error Messages
- WordPress REST API FAQ — Require Authentication
- WordPress REST API Users Endpoint

5.5 Ensure reauthentication is required for privileged actions **Profile Applicability: Level 2**

Assessment Status: Manual

Description: WordPress should require reauthentication (sudo mode) before performing sensitive administrative actions. Users should be challenged for their password (and 2FA) when they attempt destructive or high-risk operations. This can be achieved with Fortress or wp-sudo (disclosure: maintained by this document's editor).

Rationale: If a session is hijacked, reauthentication limits the damage the attacker can do with the stolen session. Gating critical operations ensures that even with a stolen browser cookie, the attacker cannot perform permanent or high-impact changes without knowing the user's password. See the WordPress Security Hardening Guide §8.2 for the enterprise architecture rationale and the full list of recommended gated actions.

Impact: Requires a dedicated security solution (e.g., Fortress by Snicco) or an equivalent platform control. WordPress core does not natively enforce reauthentication for most admin actions. Managed platforms may provide this natively — for example, WordPress VIP requires step-up authentication (MFA reauthentication) before accessing higher-risk resources or performing sensitive, irreversible VIP Dashboard actions, with a one-hour unlock window.

Audit:

This is a manual check. Verify that a reauthentication challenge is triggered for at least the following categories of actions: 1. **Plugins & Themes:** Installation, deletion, and updates. 2. **Users:** Creation, deletion, and role promotion (especially to Administrator). 3. **Authentication:** Application password creation (to prevent persistent backdoors). 4. **Configuration:** Edits to `wp-config.php` constants or critical site options. 5. **Tools:** Data export (WXR) and WordPress core updates.

Remediation:

Implement a time-bound, action-gated, or role-based reauthentication solution. Configure the "Action Registry" to gate all high-priority destructive operations. Ensure the solution supports both the Dashboard UI and relevant API surfaces (AJAX/REST, XML-RPC, etc.).

Default Value: WordPress requires password confirmation only for profile email/password changes.

References:

- OWASP Authentication Cheat Sheet — Reauthentication
- NIST SP 800-63B — Reauthentication
- Fortress sudo mode documentation (Snicco)
- WordPress VIP — Step-Up Authentication (example platform implementation)

5.6 Ensure REST API exposure is intentionally scoped **Profile Applicability: Level 2**

Assessment Status: Automated

Description: Keep required public REST endpoints available, and restrict only endpoints that should not be public (for example, user-enumeration routes or custom sensitive routes).

Rationale: WordPress REST API is designed to expose some public content endpoints without authentication. Blanket authentication requirements often break themes, plugins, and headless front ends. Security should be applied at the route and permission-callback level.

Impact: Endpoint-level restrictions require route inventory and testing, but avoid the broad compatibility breakage caused by global API blocking.

Audit:

```
$ curl -sI https://example.com/wp-json/wp/v2/posts
```

Expected for public sites: `200 OK`.

```
$ curl -s https://example.com/wp-json/wp/v2/users
```

Verify public user enumeration is blocked or minimized per site policy.

Remediation:

1. Keep public content routes required by your architecture.
2. Block or harden user-enumeration and other sensitive routes.
3. Use permission callbacks on custom endpoints.

Example (must-use plugin) to block unauthenticated users endpoints:

```
add_filter( 'rest_endpoints', function( $endpoints ) {
    if ( ! current_user_can( 'list_users' ) ) {
        unset( $endpoints['/wp/v2/users'] );
        unset( $endpoints['/wp/v2/users/(?P<id>[\d]+)'] );
    }
    return $endpoints;
} );
```

Default Value: Public REST endpoints are available by default.

References:

- WordPress REST API Authentication
- WordPress REST API FAQ
- WordPress REST API Users Endpoint

5.7 Ensure a strong password policy is enforced **Profile Applicability: Level 1**

Assessment Status: Manual

Description: Configure WordPress to enforce a strong password policy that follows current OWASP and NIST recommendations: minimum length of 15 characters as the baseline recommendation, screening against breached password and dictionary lists, and no arbitrary complexity rules that lead to predictable patterns. NIST permits 8 characters only when a password is used solely as part of a multi-factor authentication flow.

Rationale: Weak passwords are a primary vector for account takeover. Modern standards (NIST SP 800-63B and OWASP) emphasize length and entropy over complexity (e.g., forcing special characters), and mandate checking against known compromised credentials.

Impact: Users may need to update existing weak passwords. Requires a plugin for advanced enforcement.

Note on Password Hashing: As of WordPress 6.8, user passwords are hashed with bcrypt by default. Argon2id is supported on compatible PHP environments and provides stronger resistance to GPU-accelerated brute-force attacks. For high-security deployments, consider enabling Argon2id via the `wp_hash_password_algorithm` filter, typically implemented as a must-use plugin.

Audit:

This is a manual check. Verify that: 1. A password enforcement mechanism is active. 2. Test by attempting to set a simple 8-character password; verify it is rejected. 3. Verify the policy requires at least 15 characters.

Remediation:

1. Install a security plugin that supports password policy enforcement. 2. Configure the policy to require a minimum of 15 characters. 3. Enable “pwned password” checks to block credentials found in previous data breaches. 4. Remove legacy requirements for symbols or numbers if they interfere with user-generated passphrases.

Default Value: WordPress encourages strong passwords but does not strictly enforce a minimum length or check against breached lists by default.

References:

- OWASP Authentication Cheat Sheet
- NIST SP 800-63B, §3.1.1.2, and Appendix A

5.8 Ensure user roles and capabilities are defined in code **Profile Applicability: Level 2**

Assessment Status: Manual

Description: User roles and custom capabilities should be defined in code (via a must-use plugin) rather than relying solely on database-stored role definitions, and the stored definitions should be reconciled against the code. This file-based approach makes role definitions version-controlled and auditable, and makes unexpected changes detectable.

Rationale: WordPress stores role definitions in the `{prefix}user_roles` option and each user's role assignment in user meta. Both can be modified by an attacker who achieves SQL injection or gains admin access. Keeping the intended definitions in code gives a reviewable source of truth to compare against. Code alone does not protect the stored copy: `add_role()` writes to the database and does nothing when the role already exists, so calling it on every load does not undo tampering. Reconciliation (resetting each managed role's capabilities to the coded list) restores role definitions but not user-to-role assignments, which must be audited separately (see 5.2).

Impact: Requires development effort to codify custom roles. Changes to roles must go through the deployment pipeline rather than the WordPress Dashboard.

Audit:

This is a manual check. Verify that: 1. Custom roles are defined in a must-use plugin. 2. Role definitions are stored in version control. 3. Default role capabilities have been reviewed and unnecessary capabilities removed. 4. The stored capabilities of each managed role match the coded definition. Compare with `wp role list` and `wp cap list <role>`.

Remediation:

Create a must-use plugin (`wp-content/mu-plugins/custom-roles.php`) that holds the intended definitions, removes unnecessary default capabilities, and reconciles managed roles when the stored copy differs:

```
add_action( 'init', function() {
    // Remove capabilities from default roles as needed
    $editor = get_role( 'editor' );
```

```
    if ( $editor && $editor->has_cap( 'unfiltered_html' ) ) {
        $editor->remove_cap( 'unfiltered_html' );
    }

    // Managed custom roles: the coded list is the source of truth.
    $managed = array(
        'site_manager' => array(
            'name' => 'Site Manager',
            'caps' => array(
                'read'           => true,
                'manage_options' => true,
                // Add only the capabilities this role requires
            ),
        ),
    );

    foreach ( $managed as $slug => $definition ) {
        $role = get_role( $slug );

        if ( ! $role ) {
          add_role( $slug, $definition['name'], $definition['caps'] );
            continue;
        }

     // add_role() does nothing for an existing role, so compare and reset.
        if ( $role->capabilities != $definition['caps'] ) {
        error_log( "Role {$slug} differed from its coded definition and was reset."
            remove_role( $slug );
          add_role( $slug, $definition['name'], $definition['caps'] );
        }
    }
});
```

The comparison runs on every request and writes to the database only when a difference is found. Treat a logged reset as a security event to investigate, not routine noise. Test the reconciliation in staging by adding a capability to the stored role and confirming it is removed.

Default Value: Roles are stored in the `wp_options` table and editable via plugins or direct database access.

References:

- WordPress Roles and Capabilities
- `WP_Roles::add_cap()` Class Reference
- `WP_Role::add_cap()` Class Reference

5.9 Ensure public author identities do not disclose login names or email addresses
Profile Applicability: Level 1

Assessment Status: Automated

Description: Each account's display name, which WordPress publishes as the byline, and its nicename, the slug in author URLs, should not be derived from the account's login name or email address. Treat the login as a private credential, the display name as the public byline, and the slug as a URL built from the byline.

Rationale: Control 5.4 closes the routes that list accounts. It does not change the data those routes and others publish. When no first or last name is supplied, which covers every self-registration, `wp_insert_user()` sets `display_name` and `nickname` to `user_login`, and it always derives `user_nicename` from `user_login` (`dan@example.com` becomes `danexample-com`). The display name is then printed in theme bylines, RSS `<dc:creator>` and Atom `<author>`, oEmbed `author_name`, the REST users `name` field, and the name on every comment the account posts while logged in. The nicename appears in author URLs, the REST users `slug` field, and the `comment-author-{nicename}` class on those comments. An email address is the most damaging case: WordPress has accepted an account's email address in place of its username at login since version 4.5, so a published address is always a valid login identifier as well as personal data and a lookup key for breached-password lists. Email addresses remain acceptable as login names; the OWASP Authentication Cheat Sheet recommends allowing them. The defect is publishing one.

Impact: Changing a nicename changes that author's archive URL. Do not redirect the old slug to the new one, because a redirect confirms that an account with the old, email-derived slug exists. The old URL should respond exactly as an unknown author does. Changing a display name does not update comments already posted, which store a copy of the name in `comment_author`; update those separately if required.

Audit:

List the IDs of accounts whose display name or nicename is derived from their login or email address. The command prints account IDs only, never addresses:

```
$ wp eval 'foreach ( get_users( array( "fields" => array( "ID", "user_login", "user_
    $e = strtolower( $u->user_email );
```

```
   $x = array_filter( array( strtolower( $u->user_login ), $e, strstr( $e, "@", true
    $n = strtolower( trim( $u->display_name ) );
    $s = $u->user_nicename;
    $b = preg_replace( "/-\d+$/", "", $s );
    $hit = in_array( $n, $x, true ) || preg_match( "/\S+@\S+/", $n );
    foreach ( $x as $v ) {
     foreach ( array( sanitize_title( $v ), sanitize_title( mb_substr( $v, 0, 50 ) )
        $hit = $hit || $s === $d || $b === $d || ( strlen( $b ) >= 40 && 0 === strpos( $
        }
    }
    if ( $hit ) {
        echo $u->ID, PHP_EOL;
    }
}' --path=/path/to/wordpress
```

Any output is a finding. Also confirm that administrator accounts do not publish content (see 5.2); publishing accounts should hold the Author or Editor role.

Remediation:

Fix each listed account. For example:

```
$ wp user update <ID> --display_name="Dan Knauss" --user_nicename="dan-
knauss" --path=/path/to/wordpress
```

Then prevent new and updated accounts from reintroducing the problem with a must-use plugin (`wp-content/mu-plugins/public-identity.php`). It replaces an exposed display name with the account's first and last name, or a neutral generated name, and replaces an exposed slug with one built from the display name, or a random one if that is taken:

```
add_filter( 'wp_pre_insert_user_data', function ( $data, $update, $user_id, $userda
    // Core leaves user_login out of $data on updates.
   $login  = isset( $data['user_login'] ) ? $data['user_login'] : get_userdata( $us
>user_login;
    $email   = strtolower( $data['user_email'] );
   $exposed = array_filter( array( strtolower( $login ), $email, strstr( $email, '@'

   // Core derives the default slug from the first 50 characters, so compare both for
    $derived = array();
    foreach ( $exposed as $value ) {
        $derived[] = sanitize_title( $value );
```

```
        $derived[] = sanitize_title( mb_substr( $value, 0, 50 ) );
    }

    $name_exposed = function ( $name ) use ( $exposed ) {
        $name = strtolower( trim( $name ) );
     return in_array( $name, $exposed, true ) || preg_match( '/\S+@\S+/', $name );
    };
    $slug_exposed = function ( $slug ) use ( $derived ) {
     // Strip core's -N collision suffix; a shortened base can be a prefix of the der
        $base = preg_replace( '/-\d+$/', '', $slug );
        foreach ( $derived as $value ) {
        if ( $slug === $value || $base === $value || ( strlen( $base ) >= 40 && 0 === s
                return true;
            }
        }
        return false;
    };

   // Display name: never the login, the email, its local part, or anything email-
shaped.
    if ( $name_exposed( $data['display_name'] ) ) {
     $first = isset( $userdata['first_name'] ) ? trim( $userdata['first_name'] ) :
     $last  = isset( $userdata['last_name'] ) ? trim( $userdata['last_name'] ) : '';
        $full  = trim( $first . ' ' . $last );
     $data['display_name'] = ( '' !== $full && ! $name_exposed( $full ) ) ? $full : '
    }

    // Slug: never derived from the login or email.
    if ( $slug_exposed( $data['user_nicename'] ) ) {
     $slug  = mb_substr( sanitize_title( $data['display_name'] ), 0, 50 );
        $owner = '' !== $slug ? get_user_by( 'slug', $slug ) : false;
     if ( '' === $slug || $slug_exposed( $slug ) || ( $owner && (int) $owner-
>ID !== (int) $user_id ) ) {
            $slug = strtolower( wp_generate_password( 10, false ) );
        }
        $data['user_nicename'] = $slug;
    }

    return $data;
}, 10, 4 );
```

The filter runs on every account insert and update, including self-registration and profile saves. It does not change accounts that are never saved again, so run the audit and fix existing accounts as well.

Default Value: `display_name` and `nickname` default to `user_login` when no first or last name is supplied, and `user_nicename` is always derived from `user_login`.

References:

- OWASP Authentication Cheat Sheet — Usernames
- OWASP ASVS 5.0 — V14 Data Protection (14.2.6)
- GDPR Article 25 — Data protection by design and by default
- `wp_insert_user()` Function Reference
- `wp_pre_insert_user_data` Hook Reference

6.0 File System Permissions

This section covers file ownership and permission settings for the WordPress installation.

6.1 Ensure file ownership model is documented and least-privilege **Profile Applicability: Level 2**

Assessment Status: Automated

Description: Document and enforce one least-privilege ownership model per environment. Dedicated/self-managed hosts should prefer a non-web-server file owner (for example, `wp_user:www-data`). Shared/managed environments may use web-server ownership only with explicit compensating controls.

Rationale: Ownership strategy is environment-dependent. A non-web-server owner can reduce post-compromise file tampering risk, but some platforms constrain ownership. Security outcomes depend on the full control set (permissions, deployment model, update path), not ownership alone.

Impact: Teams must choose and document a supported model. Changing ownership on an existing deployment can break updates, plugin installs, and backup tooling if not tested.

Audit:

1. Verify a documented ownership model exists for each environment (production/staging/dev).

2. Verify on-host ownership matches the documented model.
3. Verify compensating controls are present when web-server ownership is used.

Example ownership checks:

```
$ stat -c '%U:%G' /path/to/wordpress/wp-config.php
$ stat -c '%U:%G' /path/to/wordpress/wp-includes/version.php
```

Remediation:

Model A (preferred on dedicated/self-managed hosts):

```
sudo chown -R wp_user:www-data /path/to/wordpress/
sudo find /path/to/wordpress/ -type d -exec chmod 750 {} \;
sudo find /path/to/wordpress/ -type f -exec chmod 640 {} \;
sudo chmod 440 /path/to/wordpress/wp-config.php
```

In this model PHP-FPM runs as `www-data` and reads files through the group, so `wp-config.php` must be group-readable (440). Mode 400 here would make the file unreadable to PHP and take the site down. Confirm with `sudo -u www-data test -r /path/to/wordpress/wp-config.php && echo readable`.

Model B (shared/managed hosting constraints):

- Keep provider-required ownership model.
- Enforce baseline permissions (755/644 or stricter where supported).
- Set `DISALLOW_FILE_EDIT` as baseline.
- If feasible, enable `DISALLOW_FILE_MODS` with an external update pipeline.

Default Value: Ownership model depends on host provisioning and control panel behavior.

References:

- WordPress Hardening — File Permissions
- WordPress Changing File Permissions

6.2 Ensure `wp-config.php` has restrictive permissions **Profile Applicability: Level 1**

Assessment Status: Automated

Description: `wp-config.php` must have the most restrictive file permissions possible. WordPress's official hardening documentation recommends 400 (owner read-only) or 440 (owner and group read-only).

The PHP process that serves the site must be able to read the file, so choose the mode from the user the PHP-FPM pool actually runs as:

- **400** applies when the PHP-FPM pool runs as the file's owner (for example, a per-site pool user).
- **440** applies when the pool runs as a different user and reads the file through its group (for example, files owned by `wp_user:www-data` with a `www-data` pool, as in 6.1 Model A). The owning group should contain only that pool's user — never share the group between sites in a shared-hosting context.
- **600** or **640** may be used temporarily when deployment scripts must write to the file; restore the read-only mode immediately after.

Rationale: `wp-config.php` contains database credentials, authentication keys, and security-sensitive configuration. Broad read permissions could expose these to other users on a shared server or to a compromised web server process. Making the file read-only (400/440) ensures it cannot be modified by any process running as the site user, providing an additional layer of integrity protection.

Impact: Legitimate automated tools, deployment scripts, or certain WordPress plugins that attempt to write to `wp-config.php` (e.g., caching plugins adding rules) will fail and require manual configuration by a system administrator.

Audit:

```
$ stat -c '%a %U:%G' /path/to/wordpress/wp-config.php
```

Verify permissions are 400 or 440 (600 or 640 are acceptable minimums where write access is required), and that the file is not writable by the web server user. Then confirm the serving PHP user can read it:

```
$ sudo -u www-data test -r /path/to/wordpress/wp-config.php && echo readable
```

Remediation:

When PHP-FPM runs as a different user than the file owner (6.1 Model A):

```
chown wp_user:www-data /path/to/wordpress/wp-config.php
chmod 440 /path/to/wordpress/wp-config.php
```

When PHP-FPM runs as the file owner:

```
chown wp_user:wp_user /path/to/wordpress/wp-config.php
chmod 400 /path/to/wordpress/wp-config.php
```

Default Value: 644 (world-readable) in many default configurations.

References:

- WordPress Changing File Permissions
- WordPress Hardening

6.3 Ensure `wp-config.php` is placed above the document root **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Where server configuration allows, `wp-config.php` should be moved one directory above the web document root. WordPress automatically detects this placement and loads the file from the parent directory.

Rationale: Placing `wp-config.php` above the document root can provide defense in depth by reducing direct web exposure. However, WordPress documentation also notes this is debated and can backfire if web-root boundaries are misconfigured.

Impact: Some hosting environments (e.g., shared hosting with restricted directory structures, some containerized setups) may not support this configuration. Additionally, if the WordPress installation is in the web root itself (rather than a subdirectory), the parent directory must not be another site's web root.

Audit:

Verify `wp-config.php` is not inside the document root:

```
$ ls -la /var/www/example.com/wp-config.php
```

If the file exists in the document root, it should be moved. Verify WordPress functions correctly after the move:

```
$ curl -sI https://example.com/ | head -5
```

Remediation:

Move `wp-config.php` one directory above the document root:

```
$ mv /var/www/example.com/wp-config.php /var/www/wp-config.php
```

WordPress will automatically detect and load `wp-config.php` from the parent directory. No additional configuration is needed.

Verify the parent directory is not publicly accessible and has restrictive permissions:

```
$ chmod 750 /var/www/
```

Default Value: `wp-config.php` is placed in the WordPress installation root (typically the document root) by default.

References:

- WordPress `wp-config.php` — Moving the File
- WordPress Hardening

7.0 Logging and Monitoring

This section addresses audit logging, activity monitoring, and intrusion detection for WordPress.

7.1 Ensure WordPress user activity logging is enabled **Profile Applicability: Level 1**

Assessment Status: Manual

Description: A WordPress audit logging solution must be installed and configured to record all user activity, including: logins and logouts, failed login attempts, content creation and modification, user account changes, plugin and theme changes, and settings modifications.

Rationale: Audit logs are essential for detecting unauthorized activity, supporting incident response, and meeting compliance requirements (PCI DSS, HIPAA, GDPR). Without logging, security incidents may go undetected and forensic analysis is impossible.

Impact: Requires a third-party plugin. Recommended: WP Activity Log, Stream, or equivalent.

Audit:

This is a manual check. Verify that: 1. An audit logging plugin is installed and active. 2. Logs capture login activity, content changes, and settings modifications. 3. Logs are retained for a period consistent with compliance requirements.

Remediation:

Install and configure an audit logging plugin (e.g., WP Activity Log). Configure log retention for at least 90 days (or per organizational policy). Enable email alerts for critical events: failed logins, new admin users, plugin changes. Export logs to a centralized SIEM for correlation (Level 2).

Default Value: No audit logging is configured by default.

References:

- OWASP Logging Cheat Sheet
- `wp_login` Hook

7.2 Ensure file integrity monitoring is configured **Profile Applicability: Level 2**

Assessment Status: Automated

Description: A mechanism should be in place to detect unauthorized changes to WordPress core files, plugins, themes, and configuration files.

Rationale: Unauthorized file modifications are a strong indicator of compromise. Integrity monitoring detects web shells, backdoors, and unauthorized code changes.

Impact: Can be implemented at the server (WP-CLI, AIDE, OSSEC, Tripwire) or application level (via many third-party plugins) or both.

Audit:

For WordPress core integrity:

```
$ wp core verify-checksums --path=/path/to/wordpress
```

For plugin integrity:

```
$ wp plugin verify-checksums --all --path=/path/to/wordpress
```

Verify both commands report no modifications.

Remediation:

1. Run `wp core verify-checksums` and `wp plugin verify-checksums` on a scheduled basis (daily recommended). 2. Install a file integrity monitoring plugin or configure server-level monitoring. 3. Alert on any unexpected file changes in `wp-includes/`, `wp-admin/`, and plugin directories.

> For AIDE installation, database initialization, automated cron scheduling, and WordPress file-change detection commands, see WordPress Operations Runbook §5.6.

Default Value: No integrity monitoring is configured by default.

References:

- WP-CLI `wp core verify-checksums`
- WP-CLI `wp plugin verify-checksums`

7.3 Ensure server-level malware detection is configured **Profile Applicability: Level 2**

Assessment Status: Manual

Description: A server-level malware detection solution should be deployed to scan the WordPress file system for known malicious patterns, web shells, backdoors, and unauthorized modifications beyond what WordPress-level integrity checks can detect.

Rationale: Application-level file integrity checks (7.2) compare files against known-good checksums but cannot detect malware injected into non-core files, uploaded web shells, or obfuscated payloads in legitimate-looking files. As well, all application-level security tools, including malware scanning provided by third-party plugins, are unreliable in an actual breach scenario, since their code executes within the compromised environment. Malware is often designed to defeat application-level scanners. Network and server-level scanning tools can reliably use signature databases and heuristic analysis that cover a broader threat surface from a secure position if the application environment is breached.

Impact: Scanning may consume server resources. Schedule intensive scans during low-traffic periods. Real-time monitoring (where available) adds minimal overhead.

Audit:

This is a manual check. Verify that: 1. A server-level malware scanning tool is installed and active (e.g., Imunify360, Linux Malware Detect, ClamAV). 2. Regular scans are scheduled (daily recommended). 3. Scan results are reviewed and alerts are configured for detections.

Remediation:

1. Install a server-level malware detection tool appropriate to the environment:
    - **Managed hosting:** Verify the hosting provider includes malware scanning (most enterprise WordPress hosts include Imunify360 or equivalent).
    - **Self-managed:** Install Linux Malware Detect (LMD) with ClamAV as a scanning engine.
2. Configure daily scheduled scans of the WordPress installation directory.
3. Enable real-time monitoring of the `wp-content/` directory if supported.
4. Configure email or SIEM alerts for malware detections.

Default Value: No malware detection is configured by default on most server environments.

References:

- ClamAV Documentation
- WordPress Hardening
- Snicco series on WordPress plugin-based malware scanners

8.0 Supply Chain and Component (Plugin and Theme) Management

This section addresses the security of pluggable WordPress components known as plugins and themes (as well as their sub-components called extensions or add-ons), and their update processes.

8.1 Ensure all unused plugins and themes are removed **Profile Applicability: Level 1**

Assessment Status: Automated

Description: All deactivated plugins and non-active themes (except one default fallback theme) should be deleted from the server, not merely deactivated.

Rationale: Deactivated plugins and themes remain on the file system and may contain exploitable vulnerabilities. PHP files in deactivated plugins can be accessed directly if the web server processes them, bypassing WordPress entirely.

Impact: Administrators cannot quickly activate previously installed but inactive themes or plugins; they must be re-downloaded and re-installed when needed.

Audit:

```
$ wp plugin list --status=inactive --fields=name,version --path=/path/to/wordpress
$ wp theme list --status=inactive --fields=name,version --path=/path/to/wordpress
```

Verify no unused plugins or themes are present (one default/fallback theme is acceptable).

Remediation:

```
$ wp plugin delete <plugin-name> --path=/path/to/wordpress
$ wp theme delete <theme-name> --path=/path/to/wordpress
```

Retain only the active theme and one default WordPress theme as a fallback.

Default Value: Default themes and example plugins (Akismet, Hello Dolly) are included in fresh installations.

References:

- WordPress Hardening
- WordPress Managing Plugins
- WordPress Managing Themes

8.2 Ensure all plugins and themes are from trusted sources **Profile Applicability: Level 1**

Assessment Status: Manual

Description: Plugins and themes should only be installed from the official WordPress.org repository or verified commercial vendors. Nulled (pirated) plugins and themes must never be used.

Rationale: Nulled and pirated plugins are a leading vector for malware distribution. They frequently contain backdoors, cryptominers, SEO spam injectors, and other malicious code. Even legitimate-appearing free plugins from unofficial sources may be trojanized.

Impact: Prevents the use of unauthorized or custom plugins that have not gone through an organizational vetting process, potentially slowing down the adoption of new features.

Audit:

This is a manual check. Review all installed plugins and themes:

```
$ wp plugin list --fields=name,status,version,update --path=/path/to/wordpress
```

Verify each plugin is available in the WordPress.org repository or from a known commercial vendor.

Remediation:

1. Audit all installed plugins and themes for their source. 2. Remove any plugins not traceable to a legitimate source. 3. Establish an approved plugin list for the organization. 4. Disable the built-in file editor by default (see 4.1: `DISALLOW_FILE_EDIT`). Use `DISALLOW_FILE_MODS` only for hardened profiles with external update workflows.

Default Value: WordPress allows installation from any ZIP file by default.

References:

- WordPress Plugin Developer Guidelines
- WordPress Hardening

8.3 Ensure plugin and theme updates are applied promptly **Profile Applicability: Level 1**

Assessment Status: Manual

Description: Security updates for plugins and themes should be applied within 72 hours of release. Critical security updates should be applied immediately or virtual-patched within 24 hours.

Rationale: Known vulnerabilities in popular WordPress plugins are actively exploited within hours of public disclosure. Delayed patching is the most common technical root cause of WordPress compromises.

Impact: Updates may occasionally introduce regressions or compatibility issues. Use a staging environment for testing when possible, but do not delay critical security patches.

Audit:

```
$ wp plugin list --fields=name,version,update --path=/path/to/wordpress
$ wp theme list --fields=name,version,update --path=/path/to/wordpress
```

Verify no security updates are pending.

Remediation:

1. Enable auto-updates for plugins and themes where supported. 2. Subscribe to vulnerability notification services (Patchstack, Sucuri, WPScan, and Wordfence). Use the Exploit Prediction Scoring System (EPSS) alongside CVSS to prioritize remediation by real-world exploitability. 3. Establish a maintenance schedule for manual update review (weekly minimum). 4. Deploy virtual patching for critical vulnerabilities that cannot be patched immediately.

> For the complete update procedure with staging verification, rollback steps, and escalation criteria, see WordPress Operations Runbook §6.3.

Default Value: Plugin and theme auto-updates are disabled by default (can be enabled per-plugin).

References:

- Patchstack Vulnerability Database
- Exploit Prediction Scoring System

8.4 Ensure a Software Bill of Materials (SBOM) is maintained **Profile Applicability: Level 2**

Assessment Status: Manual

Description: A machine-readable Software Bill of Materials (SBOM) should be maintained for each WordPress deployment, documenting all components including WordPress core version, all active and inactive plugins and themes, third-party libraries bundled within plugins or themes, PHP version and extensions, and web server and database versions.

Rationale: Supply chain compromise accounted for 15% of breaches in IBM's Cost of a Data Breach Report (2025) with an average cost of $4.91 million, and third-party involvement doubled to 30% per the Verizon DBIR (2025). An SBOM enables rapid identification of affected components when a vulnerability is disclosed in any dependency, reducing the time to assess exposure and apply patches.

Impact: Requires tooling and process to generate and maintain the SBOM. Can be automated through WP-CLI scripts and CI/CD pipeline integration.

Audit:

This is a manual check. Verify that: 1. An SBOM document or automated generation process exists for the WordPress deployment. 2. The SBOM is updated when plugins, themes, or core are added, updated, or removed. 3. The SBOM is stored in a location accessible to the security team.

Remediation:

1. Generate an SBOM using WP-CLI and system commands:

```
# Core version
wp core version --path=/path/to/wordpress

# All plugins with versions
wp plugin list --fields=name,version,status --path=/path/to/wordpress --format=json

# All themes with versions
wp theme list --fields=name,version,status --path=/path/to/wordpress --format=json

# PHP version and extensions
php -v && php -m

# Web server and database versions
nginx -v 2>&1 || apache2 -v 2>&1  # On RHEL/AlmaLinux/Rocky Linux: httpd -v
mysql --version
```

2. Integrate SBOM generation into deployment pipelines.
3. Cross-reference the SBOM against vulnerability databases (Patchstack, Sucuri, WP-Scan, and Wordfence) on a regular schedule.

Default Value: No SBOM is maintained by default.

References:

- CISA SBOM Resources
- OWASP Software Bill of Materials Cheat Sheet

9.0 Web Application Firewall

This section addresses the deployment and configuration of a Web Application Firewall (WAF) to protect the WordPress application.

9.1 Ensure Web Application Firewall is configured **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Deploy a Web Application Firewall (WAF). This can be a server-level solution, such as **ModSecurity**, **open-appsec**, **NGINX App Protect WAF**, **7G WAF**, or **Coraza** (OWASP Coraza WAF) with the OWASP Core Rule Set (CRS), or a cloud-based solution, such as **Cloudflare WAF**, **Akamai**, or **Sucuri**.

Rationale: A WAF provides immediate protection against common web attacks (SQLi, XSS, RCE) before they reach the application. For server-level WAFs (for example, ModSecurity or Coraza), WordPress-specific exclusion rules are necessary to allow legitimate functionality (like post saving and media uploads) to pass through without being blocked as false positives. Cloud WAFs typically manage these rulesets automatically.

Impact: WAFs can introduce latency and false positives. Tuning is required. Cloud WAFs may require DNS changes.

Audit:

This is a manual check. Verify that: 1. A WAF is active and blocking malicious requests (verify via logs or simulation). 2. If using a server-level WAF, the OWASP Core Rule Set and WordPress Rule Exclusions are enabled. 3. If using a Cloud WAF, the WordPress-specific protection profile is active.

Remediation:

For server-level WAF: 1. Install ModSecurity or Coraza with the OWASP Core Rule Set. 2. Enable the WordPress exclusion rule set. - For OWASP CRS v3.x: Uncomment the WordPress exclusion rule in `crs-setup.conf`. - For OWASP CRS v4.x: Use the WordPress Rule Exclusions Plugin.

For Cloud WAF: 1. Route traffic through a provider such as Cloudflare, Akamai, or Sucuri. 2. Enable Managed Rulesets related to WordPress and OWASP Top 10.

Default Value: No WAF is configured by default.

Note: While this benchmark classifies WAF as Level 2, enterprise WordPress deployments should treat WAF as a baseline requirement. See the companion WordPress Security Architecture and Hardening Guide for extended enterprise guidance.

References:

- OWASP Core Rule Set (CRS)
- WordPress Rule Exclusions Plugin for OWASP CRS

10.0 Backup and Recovery

This section addresses backup strategy, offsite storage, and recovery procedures for WordPress deployments.

10.1 Ensure backup and recovery procedures are implemented **Profile Applicability: Level 1**

Assessment Status: Manual

Description: Automated backups of the complete WordPress installation (files and database) must be performed regularly, stored offsite, encrypted, and tested for successful restoration on a recurring schedule.

Rationale: In the event of a security breach, the most reliable recovery strategy is to identify the root cause, verify the integrity of backups, and rebuild the compromised system from a known-good state. Without tested backups, recovery from ransomware, data corruption, or a complete site compromise may be impossible.

Impact: Backup processes consume storage and may briefly impact site performance during execution. Server-level backups are preferred over WordPress plugin-based backups for reliability and scope.

Audit:

This is a manual check. Verify that: 1. Automated backups are running on a defined schedule (daily minimum for most sites). 2. Backups include both the database and the full file system (WordPress core, `wp-content/`, and `wp-config.php`). 3. Backups are stored offsite, in a location inaccessible from the production server. 4. Backup data is encrypted both in transit and at rest. 5. Backup restoration has been tested within the last quarter. 6. Multiple backup generations are retained with sufficient history to recover from undetected compromises.

Remediation:

1. Configure server-level backups (not relying solely on WordPress plugins) with daily frequency at minimum.
2. Store backups in an offsite location (e.g., a separate cloud storage account, S3 bucket, or remote server) that is not accessible from the production web server.

3. Encrypt backup archives using AES-256 or equivalent.
4. Test backup restoration quarterly by performing a full recovery to a staging environment.
5. Retain at least 30 days of daily backups and 90 days of weekly backups, allowing recovery from compromises that may not be detected immediately.
6. Document the recovery procedure and assign clear ownership and responsibilities.

Default Value: No backups are configured by default. Backup responsibility depends on the hosting environment.

References:

- WordPress Backups
- WordPress Backing Up Your Database
- WordPress Backing Up Your Files

11.0 AI Integration Security

AI tools are increasingly integrated into WordPress workflows for content generation, chatbots, code assistance, and site management. IBM's Cost of a Data Breach Report (2025) found that 13% of organizations experienced a breach involving an AI model or application, and 97% of those breaches involved systems lacking proper access controls. This section provides controls for securing AI integrations in WordPress environments.

11.1 Ensure AI API keys are securely stored **Profile Applicability: Level 1**

Assessment Status: Automated

Description: API keys and credentials for AI/LLM services (OpenAI, Anthropic, Google, etc.) must be stored securely and never exposed in client-side code, version control, or the WordPress database.

Rationale: AI API keys grant access to paid services and may allow data exfiltration or abuse. The Verizon DBIR (2025) found that leaked secrets in code repositories had a median remediation time of 94 days, with 66% being JSON Web Tokens. AI API keys are similarly at risk. The IBM X-Force 2026 Threat Intelligence Index reports that stolen credentials for AI chatbot platforms are now appearing in underground marketplaces, driven largely by infostealer infections — making secure storage and rotation of AI API keys an operational priority, not just a best practice.

Impact: Requires using `wp-config.php` constants or environment variables rather than storing keys in plugin settings or database-backed connector settings. In WordPress 7.0, the Connectors API can source AI provider credentials from environment variables, PHP constants, or the database, but database-backed connector secrets are not encrypted by core.

Audit:

1. Search the codebase and database for AI service API keys:

```
# List file names only (-l); do not print matching lines, which contain the keys.
grep -rl "sk-" /path/to/wordpress/wp-content/ --include="*.php"
# Return option names only; do not select option_value.
wp db query "SELECT option_name FROM $(wp db prefix)options WHERE option_value LIKE
%' OR option_value LIKE '%key-%'"
# Run as the WordPress site user, not root.
```

These commands report where possible keys are stored without copying the key values into terminal history, logs, tickets, or AI-assisted sessions. 2. Verify API keys are defined as constants in `wp-config.php` or loaded from environment variables rather than relying on database-backed connector settings. 3. Confirm `.gitignore` excludes `wp-config.php` and environment files.

Remediation:

1. Move all AI API keys to `wp-config.php` constants (e.g., `define('OPENAI_API_KEY', getenv('OPENAI_API_KEY'));`).
2. Remove any API keys stored in the `wp_options` table, plugin settings screens, or database-backed connector settings.
3. Rotate any keys that have been exposed in version control or client-side code.

Default Value: Many AI plugins — and WordPress 7.0 connector setups that rely on the admin screen instead of environment variables or PHP constants — store API keys in the database.

References:

- OWASP Secrets Management Cheat Sheet
- Introducing the Connectors API in WordPress 7.0
- Felix Arntz: Storing Confidential Data in WordPress
- Pantheon: Secrets Management in WordPress
- Snicco: Vaults and Pillars
- WordPress VIP Environment Variables
- WordPress VIP `vip-config.php`

11.2 Ensure AI-generated content is sanitized **Profile Applicability: Level 1**

Assessment Status: Manual

Description: All content generated by AI services must be treated as untrusted input and sanitized before rendering or storage, using WordPress's standard escaping functions (`esc_html()`, `wp_kses()`, `esc_attr()`, etc.).

Rationale: AI models can generate content that includes XSS payloads, SQL injection patterns, or malicious HTML — either through prompt injection attacks or as a natural consequence of model behavior. Treating AI output as trusted input bypasses WordPress's defense-in-depth sanitization model.

Impact: Minimal. Requires using existing WordPress sanitization functions on AI output, which is consistent with standard development practice for any external data source.

Audit:

Review custom AI integration code for direct output of AI-generated content without sanitization. Check that AI responses pass through WordPress escaping functions before being stored in post content, meta fields, or rendered in templates.

Remediation:

1. Apply `wp_kses_post()` to AI-generated content before storing in `post_content`.
2. Apply `esc_html()` or `esc_attr()` before rendering in HTML templates.
3. Never pass AI-generated content directly to `$wpdb->query()` without `$wpdb->prepare()`.

Default Value: No WordPress-specific defaults; depends on plugin implementation.

References:

- WordPress Sanitizing Data
- WordPress Escaping Data
- `wp_kses_data()` Function Reference

11.3 Ensure AI tool usage is governed by policy **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Organizations must maintain and enforce a policy governing the use of AI tools by WordPress team members and integrated into WordPress sites. The policy must address approved tools, data classification for AI inputs, authentication requirements, and disclosure of AI-generated content.

Rationale: Shadow AI — the unsanctioned use of AI tools by employees — is a measurable risk. IBM's Cost of a Data Breach Report (2025) found that 20% of breached organizations experienced a shadow AI incident, adding $200,000 to average breach costs ($670,000 for organizations with high shadow AI prevalence), and that 63% of organizations lack AI governance policies. The Verizon DBIR (2025) found that 15% of employees routinely access GenAI systems on corporate devices, with 72% using non-corporate email accounts.

Impact: Requires organizational policy development and enforcement. May restrict which AI plugins can be installed on WordPress sites.

Audit:

1. Verify that a written AI acceptable use policy exists and has been communicated to all WordPress team members.
2. Confirm that only approved AI plugins are installed on production sites.
3. Verify that AI service authentication uses corporate SSO or managed API keys (not personal accounts).

Remediation:

1. Develop an AI acceptable use policy covering approved tools, prohibited data inputs (credentials, PII, proprietary content), and disclosure requirements.
2. Maintain an inventory of approved AI plugins and services.
3. Configure AI services with corporate authentication and audit logging where available.
4. Include AI governance in security awareness training.

Default Value: No AI governance policy exists by default.

References:

- NIST AI Risk Management Framework Playbook
- OWASP Top 10 for Large Language Model Applications

11.4 Ensure Abilities API authorization overrides are reviewed **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Where the Abilities API is used (WordPress 6.9 and later; client-side integration from 7.0), code that can bypass or override an ability's permission check must be inventoried and reviewed, and every ability exposed to external clients must enforce authorization in its own `permission_callback`.

Rationale: Abilities are named operations that AI agents, automation tools, and plugins can discover and run. WordPress 7.1 added execution lifecycle filters that any active plugin or theme can hook. Two of them change authorization outcomes. `wp_ability_permission_result` can turn a denial from the ability's own `permission_callback` into an allow on every execution path, including the REST API; a callback that returns `true` lets even an unauthenticated request run a REST-exposed ability. `wp_pre_execute_ability` can return a result before input validation and the permission check run when code calls `WP_Ability::execute()` directly from PHP; it does not bypass the check on the core REST run endpoint, which validates input and checks permissions in its own permission callback before `execute()` is called. The same split holds for the other channels: WP-CLI's `wp ability run` calls `execute()` directly, so the filter bypasses the check there, while the MCP Adapter (0.7.0) checks permissions before executing, so it does not. WordPress 7.1 also added a `public` metadata flag that controls whether an ability is exposed to external clients such as the REST API. That flag, like `show_in_rest`, controls discovery and exposure only; the core dev note states that exposure flags must not be treated as a security boundary. In WordPress 7.1.3, the REST list endpoints require a logged-in user with the `read` capability and show every REST-exposed ability to any such user, whether or not that user may run it; abilities that are not REST-exposed return 404 on the REST list and run endpoints but remain executable from PHP. Exposure is decided per channel. The MCP Adapter (0.7.0) uses `meta.mcp.public` when set and otherwise inherits `public`, and ignores `show_in_rest`, so an ability hidden from REST with `show_in_rest => false` can still be exposed to MCP clients. WP-CLI (`wp ability list` and `wp ability run`) applies no exposure check: it lists every registered ability and runs any of them, subject to the permission check for the `--user` given.

Impact: Requires a code inventory when plugins are added or updated. No runtime impact.

Audit:

This is a manual check.

1. Find code that hooks the authorization-affecting filters:

```
$ grep -rnE "wp_pre_execute_ability|wp_ability_permission_result" /path/to/wordpre
content/ --include="*.php"
```

Review each match. A callback on `wp_ability_permission_result` that can return `true`, or a callback on `wp_pre_execute_ability` that returns a result, must make its own equivalent authorization decision and have a documented reason.

2. List the abilities exposed through the REST API, as an authenticated administrator (for example, with an application password belonging to a dedicated audit account):

```
$ curl -s -u audit-user:APPLICATION-PASSWORD https://example.com/wp-
json/wp-abilities/v1/abilities
```

Where the `wp ability` command is installed (the `wp-cli/ability-command` package), list every registered ability with its exposure flags, including those hidden from REST:

```
# Plugin-dependent: requires the wp-cli/ability-command package.
$ wp ability list --fields=name,public,show_in_rest
```

If the MCP Adapter plugin is active, also review abilities that set `meta.mcp.public`. For each exposed ability, confirm the registering code supplies a `permission_callback` that checks a capability appropriate to what the ability does. A callback that only returns `true`, or only checks that the user is logged in, is a finding unless the ability returns non-sensitive data.

3. Confirm that logging attached to `wp_ability_invoked` does not record raw input. The action fires before validation and receives unfiltered input, which may contain secrets or personal data. Do not rely on it as a complete record of denied attempts: it fires inside `WP_Ability::execute()`, which the REST run endpoint and the MCP Adapter never reach for a request they reject. Denied WP-CLI runs are recorded, because WP-CLI calls `execute()` directly.

Remediation:

Remove or correct authorization overrides that are not justified. Add or tighten the `permission_callback` on each exposed ability. For abilities that should not be available to external clients, leave `public` unset (it defaults to `false`) and do not set `show_in_rest`. Give AI agents and automation tools a dedicated least-privilege account rather than an administrator's credentials (see 11.3).

Default Value: Abilities are not exposed to external clients unless `public` or `show_in_rest` is set. WordPress core registers no callbacks on the lifecycle filters.

The three core abilities are REST-exposed; `core/get-site-info` and `core/get-environment-info` require `manage_options`, and `core/get-user-info` is available to any logged-in user for their own account.

Behavior described in this control was verified on October 7, 2026 against WordPress 7.1.3 core code, the `wp-cli/ability-command` package (commit `c6112cc`), and MCP Adapter 0.7.0, on a test site.

References:

- New execution lifecycle filters for the Abilities API in WordPress 7.1
- A unified public exposure flag for abilities in WordPress 7.1
- Abilities API improvements in WordPress 7.1

12.0 Server Access and Network

This section addresses secure remote access to the server hosting WordPress and host-level network controls.

12.1 Ensure SSH key-based authentication is enforced **Profile Applicability: Level 1**

Assessment Status: Automated

Description: SSH access to the server must use key-based authentication only. Password-based SSH authentication must be disabled.

Rationale: Password-based SSH authentication is vulnerable to brute-force and credential stuffing attacks. Key-based authentication is significantly more resistant to these attacks and is the standard for secure server access.

Impact: All administrators must generate and deploy SSH key pairs before password authentication is disabled. Emergency access procedures should be documented.

Audit:

Read the effective configuration, which includes `Include` files and defaults that a search of `sshd_config` misses:

```
$ sudo sshd -T | grep -Ei '^(passwordauthentication|kbdinteractiveauthentication|pu
```

Verify: `passwordauthentication no`, `kbdinteractiveauthentication no`, and `pubkeyauthentication yes`. If the configuration uses `Match` blocks, repeat for the relevant context (for example, `sudo sshd -T -C user=deploy,host=example.com,addr=203.0.113.1`

Remediation:

In `/etc/ssh/sshd_config`:

```
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
PermitRootLogin no
```

`PasswordAuthentication no` alone does not close password logins that PAM offers through keyboard-interactive authentication. Where an approved design combines keys with a one-time code through PAM, keep keyboard-interactive enabled and require both with `AuthenticationMethods publickey,keyboard-interactive`.

Validate the configuration, then restart the SSH service:

```
$ sudo sshd -t
$ sudo systemctl restart sshd
```

Confirm a new key-based session connects before closing the existing one.

Default Value: Password authentication is enabled by default on most Linux distributions.

References:

- OpenSSH `sshd_config` Manual

12.2 Ensure SFTP is used and FTP is disabled **Profile Applicability: Level 1**

Assessment Status: Automated

Description: Prefer SFTP (SSH File Transfer Protocol) or SCP for file transfers. Plain FTP must be disabled. FTPS may be permitted temporarily for legacy interoperability, with enforced TLS and a migration plan to SFTP.

Rationale: FTP transmits credentials and data in cleartext and should not be used. SFTP operates over SSH and is simpler to secure operationally. FTPS can be secured, but typically introduces higher operational complexity than SFTP.

Impact: Users and deployment tools relying on FTP must be migrated to SFTP. Most modern FTP clients support SFTP natively.

Audit:

```
$ ss -tlnp | grep -E ':21\b'
```

Verify no service is listening on port 21.

```
$ dpkg -l | grep -iE 'vsftpd|proftpd|pure-ftpd'
```

Verify no FTP server packages are installed.

Remediation:

1. Remove any installed FTP server software:

```
$ sudo apt purge vsftpd proftpd-basic pure-ftpd 2>/dev/null
```

2. Verify SFTP is available through the SSH server (enabled by default in OpenSSH):

```
$ grep 'Subsystem.*sftp' /etc/ssh/sshd_config
```

3. Configure deployment pipelines and team workflows to use SFTP or SCP exclusively.

Default Value: FTP is not installed by default on most modern Linux distributions, but may be present in legacy environments.

References:

- OpenSSH `sshd_config` Manual — SFTP Subsystem

12.3 Ensure a host-based firewall is configured **Profile Applicability: Level 1**

Assessment Status: Automated

Description: A host-based firewall (e.g., UFW on Ubuntu/Debian, firewalld on RHEL/CentOS) must be enabled and configured to restrict inbound traffic to only required ports: HTTP (80), HTTPS (443), and SSH.

Rationale: A host-based firewall provides a final layer of network defense, blocking unauthorized access to services running on the server even if upstream network controls fail. Changing SSH to a non-standard port is optional and mainly reduces scanning noise.

Impact: Firewall misconfiguration can lock out administrators; validate SSH access before applying deny-by-default rules.

Audit:

For UFW:

```
$ sudo ufw status verbose
```

Verify the firewall is active and only the required ports are open.

Remediation:

For UFW on Ubuntu/Debian:

```
# Enable UFW
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow required ports
sudo ufw allow 443/tcp
sudo ufw allow 80/tcp
sudo ufw allow <custom-ssh-port>/tcp

# Enable the firewall
sudo ufw enable
```

Default Value: UFW is installed but inactive on Ubuntu. No firewall is configured by default on most distributions.

References:

- Ubuntu UFW Documentation

12.4 Ensure per-site process isolation is configured **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Each WordPress site on the server should run under a dedicated system user account with its own PHP-FPM pool. Sites must not share the same PHP process user.

Rationale: Process isolation limits the blast radius of a compromise. If one site is breached, the attacker cannot read files, access databases, or modify code belonging to other sites on the same server. This is especially critical in multi-tenant hosting environments.

Impact: Requires per-site PHP-FPM pool configuration and dedicated system user accounts. Increases server resource usage proportionally to the number of sites.

Audit:

```
$ ps aux | grep php-fpm | grep -v grep
```

Verify each site's PHP-FPM pool runs as a different system user.

```
$ grep -r '^user' /etc/php/*/fpm/pool.d/
```

Verify each pool configuration specifies a unique user.

Remediation:

1. Create a dedicated system user for each site:

```
$ sudo useradd -r -s /usr/sbin/nologin site1_user
```

2. Configure a dedicated PHP-FPM pool for each site in `/etc/php/[CUSTOMIZE: 8.x]/fpm/pool.d/site1.conf`:

```
[site1]
user = site1_user
group = site1_user
listen = /run/php/php[CUSTOMIZE: 8.x]-fpm-site1.sock
listen.owner = www-data
listen.group = www-data
```

3. Set file ownership accordingly and restart PHP-FPM.

Default Value: A single `www-data` pool serves all sites by default.

References:

- PHP-FPM Pool Configuration

13.0 Multisite Security

This section addresses security considerations specific to WordPress Multisite installations.

13.1 Ensure Super Admin accounts are minimized and audited **Profile Applicability: Level 1**

Assessment Status: Manual

Description: In a WordPress Multisite network, the number of Super Admin accounts must be limited to the absolute minimum required. Super Admin grants unrestricted access across all sites in the network, including the ability to install plugins and themes network-wide, create and delete sites, and manage all users across all sites.

Rationale: Compromising any single Super Admin account grants an attacker full control over every site in the Multisite network. Unlike single-site Administrator accounts, Super Admin access cannot be scoped or restricted — it is all-or-nothing across the entire network.

Impact: Operations that require Super Admin access must be performed by a small, audited group. Day-to-day site administration should use per-site Administrator roles.

Audit:

```
$ wp super-admin list --path=/path/to/wordpress
```

Review the list. Verify that each Super Admin account is actively needed and assigned to a specific individual.

Remediation:

1. Audit existing Super Admin accounts.
2. Remove Super Admin privileges from accounts that don't require network-level access:

```
$ wp super-admin remove <username> --path=/path/to/wordpress
```

3. Assign per-site Administrator roles for day-to-day operations.
4. Review Super Admin accounts quarterly.

Default Value: One Super Admin account is created during Multisite installation.

References:

- WordPress Multisite Network Administration
- WordPress Network Admin Screen
- WP-CLI `super-admin` Command

13.2 Ensure network-activated plugins are reviewed for cross-site impact **Profile Applicability: Level 2**

Assessment Status: Manual

Description: Plugins activated at the network level in a WordPress Multisite installation run on all sites in the network. Each network-activated plugin should be reviewed for its security impact across all sites, and network activation should be reserved for plugins that genuinely require network-wide operation.

Rationale: A vulnerability in a network-activated plugin affects every site in the Multisite network simultaneously. Shared database tables and a common file system mean that a single exploit can traverse all sites. Limiting network activation reduces the shared attack surface.

Impact: Plugins that are only needed on specific sites must be activated per-site rather than network-wide. This may require changes to existing deployment workflows.

Audit:

This is a manual check. Review the list of network-activated plugins:

```
$ wp plugin list --status=active-network --fields=name,version --path=/path/to/wordpress
```

For each network-activated plugin, verify that network-wide activation is necessary rather than per-site activation.

Remediation:

1. Review all network-activated plugins.
2. Deactivate plugins at the network level that don't require network-wide operation.
3. Activate them per-site only where needed.
4. Ensure domain mapping (if used) enforces TLS across all mapped domains.

Default Value: No plugins are network-activated by default.

References:

- WordPress Multisite Plugin Management
- WordPress Network Admin Screen

Appendix A: Recommendation Summary

The following table summarizes all recommendations in this benchmark.

ID	Recommendation	Level	Assessment
1.1	Ensure TLS 1.2+ is enforced	L1	Automated
1.2	Ensure HTTP security headers are configured	L1	Automated
1.3	Ensure server tokens and version info are hidden	L1	Automated
1.4	Ensure PHP execution is blocked in uploads	L1	Automated
1.5	Ensure rate limiting is configured for all APIs	L1	Automated
2.1	Ensure expose_php is disabled	L1	Automated
2.2	Ensure display_errors is disabled	L1	Automated
2.3	Ensure dangerous PHP functions are disabled	L1	Automated
2.4	Ensure open_basedir restricts file access	L2	Automated
2.5	Ensure PHP session security is configured	L1	Automated
3.1	Ensure DB user has minimal privileges	L1	Automated
3.2	Ensure DB is not externally accessible	L1	Automated
3.3	Review table prefix strategy (optional)	L2	Manual
3.4	Ensure database query logging is enabled	L2	Automated
4.1	Ensure DISALLOW_FILE_EDIT is true (baseline)	L1	Automated
4.2	Ensure FORCE_SSL_ADMIN is true	L1	Automated
4.3	Ensure debug mode is disabled	L1	Automated
4.4	Ensure XML-RPC is disabled	L1	Automated
4.5	Ensure automatic core updates are enabled	L1	Automated
4.6	Ensure unique auth keys and salts are configured	L1	Automated

ID	Recommendation	Level	Assessment
4.7	Consider replacing wp-cron.php with system cron	L2	Automated
5.1	Ensure 2FA is required for administrators	L1	Manual
5.2	Ensure admin accounts are minimized	L1	Manual
5.3	Ensure max session lifetime is enforced	L1	Automated
5.4	Ensure user enumeration is prevented	L1	Automated
5.5	Ensure reauthentication for privileged actions	L2	Manual
5.6	Ensure REST API exposure is intentionally scoped	L2	Automated
5.7	Ensure strong password policy is enforced	L1	Manual
5.8	Ensure roles and capabilities are defined in code	L2	Manual
5.9	Ensure author identities do not disclose logins	L1	Automated
6.1	Ensure file ownership model is least-privilege	L2	Automated
6.2	Ensure `wp-config.php` has restrictive permissions	L1	Automated
6.3	Ensure `wp-config.php` is above document root	L2	Manual
7.1	Ensure user activity logging is enabled	L1	Manual
7.2	Ensure file integrity monitoring is configured	L2	Automated
7.3	Ensure server-level malware detection is configured	L2	Manual
8.1	Ensure unused plugins and themes are removed	L1	Automated
8.2	Ensure plugins and themes are from trusted sources	L1	Manual
8.3	Ensure plugin/theme updates are applied promptly	L1	Manual
8.4	Ensure a Software Bill of Materials is maintained	L2	Manual
9.1	Ensure Web Application Firewall is configured	L2	Manual
10.1	Ensure backup and recovery procedures are implemented	L1	Manual
11.1	Ensure AI API keys are securely stored	L1	Automated
11.2	Ensure AI-generated content is sanitized	L1	Manual
11.3	Ensure AI tool usage is governed by policy	L2	Manual
11.4	Ensure Abilities API authorization overrides are reviewed	L2	Manual
12.1	Ensure SSH key-based authentication is enforced	L1	Automated
12.2	Ensure SFTP is used and FTP is disabled	L1	Automated

ID	Recommendation	Level	Assessment
12.3	Ensure a host-based firewall is configured	L1	Automated
12.4	Ensure per-site process isolation is configured	L2	Manual
13.1	Ensure Super Admin accounts are minimized and audited	L1	Manual
13.2	Ensure network plugins are reviewed for cross-site impact	L2	Manual

Cross-Document Control Classification Matrix

Use this matrix to keep this benchmark aligned with the Hardening Guide and Operations Runbook.

Control Area	Baseline	Optional Hardened	Environment-Specific
File editor/mods	`DISALLOW_FILE_EDIT = true`	`DISALLOW_FILE_MODS = true` with external update pipeline	Dashboard updates retained where platform-managed patch cadence requires it
REST API	Public content routes allowed; sensitive routes protected by auth/permissions	Block user-enumeration routes and harden custom endpoint callbacks	Global unauthenticated blocking only for private/intranet deployments
XML-RPC	Keep only when required by integrations	Disable with `xmlrpc_enabled` and/or server-level block when unused	Route/IP allowlisting for required integrations
File ownership	Documented least-privilege model per environment	Per-site process isolation and immutable deploy artifacts	Provider-constrained ownership with compensating controls

Control Area	Baseline	Optional Hardened	Environment-Specific
SSH access	Key-only auth + host firewall/fail2ban	Non-standard SSH port for scanner-noise reduction	Managed-host controls where SSH is unavailable

Appendix B: Deprecated and Invalid Constants Guardrail

The following symbols should **not** be used in benchmark remediation snippets:

- `FORCE_SSL_LOGIN` (deprecated)
- `DISALLOW_PLUGIN_EDITING` (not a WordPress core constant)
- `DISALLOW_PLUGIN_ACTIVATION` (not a WordPress core constant)
- `SECURE_LOGGED_IN_COOKIE` (not a WordPress core constant)
- `define( 'XMLRPC_REQUEST', false );` (`XMLRPC_REQUEST` is a request-context constant, not a config toggle)

When documenting Argon2 support, refer to the `wp_hash_password_algorithm` filter.

Related Documents

- **WordPress Security Architecture and Hardening Guide** — Enterprise-focused security architecture and hardening guide covering threat landscape, OWASP Top 10 coverage, server hardening, authentication, supply chain, incident response, and AI security. This benchmark's technical controls complement the Hardening Guide's broader strategic guidance.
- **WordPress Security Style Guide** — Principles, terminology, and formatting conventions for writing about WordPress security.
- **WordPress Operations Runbook** — Operational procedures template for WordPress sysadmins and SREs, covering deployment, maintenance, backup, incident response, and disaster recovery.
- **WordPress Advanced Administration: Security** — The official WordPress documentation for WordPress security, covering hardening, brute-force protection, HTTPS, backups, monitoring, and multi-factor authentication.
- **WordPress Security White Paper** — The official upstream document describing WordPress core security architecture, maintained at WordPress.org.

License and Attribution

This document is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC-BY-SA-4.0). You may copy, redistribute, remix, transform, and build upon this material for any purpose, including commercial use, provided you give appropriate credit and distribute your contributions under the same license.

Sources and Acknowledgments: The format of this benchmark is adapted from industry-standard benchmarks published by the Center for Internet Security (CIS). Technical guidance draws on the OWASP Top 10 (2025), NIST SP 800-63B, the Verizon Data Breach Investigations Report (2025), and IBM's Cost of a Data Breach Report (2025).